# PENERAPAN LIGHT GRADIENT BOOSTING MACHINE UNTUK PERAMALAN HARGA KAMAR BERBASIS DERET WAKTU

**(Studi Kasus : PT Genius Utama Indonesia)**

**LAPORAN KERJA PRAKTIK**


**RAMA PANJI NARARENDRA CAHAYA**

**5231811033**

**PROGRAM STUDI SAINS DATA**

**FAKULTAS SAINS & TEKNOLOGI**

**UNIVERSITAS TEKNOLOGI YOGYAKARTA**


**2026**

# KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa yang telah melimpahkan rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan proposal kerja praktik dengan judul "**Penerapan Light Gradient Boosting Machine Untuk Peramalan Harga Kamar Berbasis Deret Waktu (Studi Kasus: PT Genius Utama Indonesia)"** sebagai bentuk pertanggungjawaban penulis terhadap mata kuliah Kerja Praktik.

Penyusunan proposal Kerja Praktik ini diajukan sebagai salah satu syarat untuk mendapatkan gelar Sarjana Sains Data dari Program Studi Sains Data Fakultas Sains & Teknologi Universitas Teknologi Yogyakarta. Keberhasilan penulis dalam penyusunan ini tidak lepas dari dukungan, bimbingan, serta bantuan dari berbagai pihak. Oleh karena itu, pada kesempatan ini penulis ingin mengucapkan terima kasih yang setulus-tulusnya kepada:

1. Bapak Dr. Bambang Moertono Setiawan, M.M., Akt., C.A., selaku Rektor Universitas Teknologi Yogyakarta.
2. Ibu Dr. Endy Marlina, S.T., M.T., selaku Dekan Fakultas Sains dan Teknologi, Universitas Teknologi Yogyakarta.
3. Ibu Sri Wulandari, S.Kom., M.Cs., selaku Ketua Program Studi Sains Data, Universitas Teknologi Yogyakarta.
4. Bapak Edi Rahmadi selaku *Digital & Revenue* di PT Genius Utama Indonesia sekaligus pembimbing lapangan yang telah memberikan kesempatan, data, serta bimbingan teknis yang sangat berharga dalam penyusunan proposal ini.
5. Bapak M. Kadri, S.T. dan Bapak Andrian Yosaredo selaku jajaran manajemen PT Genius Utama Indonesia yang telah memberikan izin pelaksanaan kerja praktik.
6. Bapak dan Ibu Dosen serta Staf Program Studi Sains Data yang telah memberikan bekal ilmu pengetahuan dan pelayanan administrasi selama masa perkuliahan.

7. Orang tua dan keluarga tercinta yang senantiasa memberikan doa yang tulus, semangat, dan dukungan tak terhingga dalam setiap langkah penulis.
8. Rekan-rekan seperjuangan mahasiswa Sains Data Universitas Teknologi Yogyakarta atas diskusi, kerja sama, dan motivasi yang diberikan.

Atas segala doa dan bantuan serta dukungan yang telah diberikan oleh seluruh pihak terkait, penulis dapat menyelesaikan proposal ini. Penulis menyadari bahwa proposal ini masih jauh dari kesempurnaan, baik dari segi penyusunan maupun materi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun.

Demikian, semoga proposal kerja praktik ini dapat diterima sebagai gagasan yang menambah kekayaan intelektual khususnya dalam bidang *Sains Data* serta bermanfaat bagi pembaca, instansi, dan penulis sendiri.

Yogyakarta, 30 Januari 2026,

Penulis

# DAFTAR ISI

KATA PENGANTAR ..... i
DAFTAR ISI ..... iii
DAFTAR TABEL ..... v
DAFTAR GAMBAR ..... vi
DAFTAR SINGKATAN ..... vii
DAFTAR ISTILAH ..... viii
DAFTAR PERSAMAAN ..... ix
BAB I PENDAHULUAN ..... 1
1.1 Latar Belakang ..... 1
1.2 Rumusan Masalah ..... 1
1.3 Batasan Masalah ..... 2
1.4 Tujuan Kerja Praktik ..... 3
1.5 Manfaat Kerja Praktik ..... 3
1.6 Sistemika Penulisan ..... 4
BAB II KAJIAN HASIL PENELITIAN DAN LANDASAN TEORI ..... 6
2.1 Kajian Hasil Penelitian ..... 6
2.2 Landasan Teori ..... 13
2.2.1 Manajemen Pendapatan Hotel ..... 13
*2.2.2 Time Series Forecasting* ..... 13
2.2.3 Decision Tree ..... 17
*2.2.4 Gradient Boosting* ..... 25
2.2.5 *Light Gradient Boosting Machine* (LightGBM) ..... 40
2.2.6 Evaluasi Kinerja Model ..... 59
BAB III TINJAUAN UMUM INSTANSI ..... 78
3.1 Gambaran Umum ..... 78
3.2 Struktur Organisasi ..... 79
3.3 Proses Bisnis / Business Rule ..... 81
BAB IV METODE KERJA PRAKTIK ..... 82
4.1 Tahapan Kerja Praktik ..... 82
4.1.1 Identifikasi Masalah ..... 82
4.1.2 Pengumpulan Data ..... 83
4.1.3 *Pre-processing* Data ..... 83
4.1.4 Rekayasa Fitur (*Feature Engineering*) ..... 84
4.1.5 Implementasi Model ..... 84
4.1.6 Evaluasi Model ..... 84
4.1.7 Visualisasi Data dan Hasil Akhir ..... 85
4.2 Data ..... 85

4.3 Jadwal Kerja Praktik ..... 86
DAFTAR PUSTAKA ..... 87
LAMPIRAN ..... 89

# DAFTAR TABEL

Tabel 4.1 Jadwal Kerja Praktik .....86

# DAFTAR GAMBAR

Gambar 3.1 Kaluku Gili Resort – PT Genius Utama Indonesia .....79
Gambar 3.2 Struktur Organisasi PT Genius Utama Indonesia .....80
Gambar 3.3 Proses Bisnis Penetapan Harga Kamar ..... **Error! Bookmark not defined.**
Gambar 4.1 Tahapan Kerja Praktik.....82

# DAFTAR SINGKATAN

| **Singkatan** | **Kepanjangan** |
|---|---|
| ADR | *Average Daily Rate* |
| API | *Application Programming Interface* |
| ARR | *Average Room Rate* |
| ARIMA | *AutoRegressive Integrated Moving* |
| EFB | *Exclusive Feature Bundling* |
| GHM | *Genius Hospitality Management* |
| GOSS | *Gradient-based One-Side Sampling* |
| KDD | *Knowledge Discovery in Database* |
| LightGBM | *Light Gradient Boosting Machine* |
| MAE | *Mean Absolute Error* |
| MAPE | *Mean Absolute Percentage Error* |
| MSE | *Mean Squared Error* |
| OTA | *Online Travel Agent* |
| PT | Perseroan Terbatas |
| RMSE | *Root Mean Squared Error* |
| XGBoost | *eXtreme Gradient Boosting* |

# DAFTAR ISTILAH

| Istilah | Keterangan / Definisi |
|---|---|
| *Booking Channel* | Saluran atau platform pemesanan kamar hotel secara daring |
| *Data Cleaning* | Proses mendeteksi dan memperbaiki (atau menghapus) data yang rusak atau tidak akurat dari dataset. |
| *Dynamic Pricing* | Strategi penetapan harga fleksibel di mana harga disesuaikan secara *real-time* berdasarkan permintaan pasar. |
| *Feature Engineering* | Proses menggunakan pengetahuan domain untuk mengekstrak fitur baru dari data mentah guna meningkatkan kinerja algoritma machine learning. |
| *Forecasting* | Proses memproyeksikan nilai masa depan berdasarkan analisis data historis dan tren saat ini. |
| *High Season* | Periode waktu tertentu dalam satu tahun di mana permintaan terhadap akomodasi wisata meningkat signifikan. |
| *Lag* | Fitur dalam analisis deret waktu yang merepresentasikan nilai variabel pada periode waktu sebelumnya. |
| *Leaf*-wise | Strategi pertumbuhan pohon keputusan pada LightGBM yang memprioritaskan pembelahan pada daun dengan *loss* terbesar. |
| *Occupancy* | Tingkat hunian kamar hotel pada periode tertentu. |
| *Outlier* | Data pencilan atau nilai yang menyimpang jauh dari pola umum data lainnya. |
| *Overfitting* | Kondisi di mana model *machine learning* terlalu menyesuaikan diri dengan data latih sehingga performanya buruk pada data baru. |
| *Property* ID | Identitas unik berbentuk numerik yang mewakili satu unit properti atau hotel dalam sistem basis data. |
| *Revenue Management* | Penerapan analitik untuk memprediksi perilaku konsumen dan mengoptimalkan ketersediaan serta harga produk untuk memaksimalkan pendapatan. |
| *Rolling Window* | Teknik statistik yang menghitung metrik (seperti rata-rata) pada sekumpulan data dalam jendela waktu yang bergerak. |
| *Room ID* | Identitas unik berbentuk numerik yang mewakili tipe kamar spesifik dalam sebuah properti. |
| *Seasonality* | Pola berulang pada data deret waktu yang muncul secara periodik dalam interval waktu tertentu, seperti mingguan, bulanan, atau tahunan. |
| *Time Series* | Rangkaian data yang dikumpulkan, dicatat, atau diamati pada interval waktu yang berurutan. |
| *Trend* | Arah pergerakan jangka panjang pada data deret waktu yang menunjukkan kecenderungan meningkat, menurun, atau stabil. |

# DAFTAR PERSAMAAN

(2.1) Persamaan Pendapatan ARR .....**Error! Bookmark not defined.**
(2.2) Persamaan Dekomposisi Deret Waktu .....**Error! Bookmark not defined.**
(2.3) Representasi Node pada *Decision Tree* .....**Error! Bookmark not defined.**
(2.4) Fungsi Kerugian *Mean Squared Error* .....**Error! Bookmark not defined.**
(2.5) Persamaan *Pseudo-residual* .....**Error! Bookmark not defined.**
(2.6) Optimasi Bobot *Leaf* pada *Gradient Boosting* ..... **Error! Bookmark not defined.**
(2.7) Pembaruan Model *Gradient Boosting* .....**Error! Bookmark not defined.**
(2.8) Model Integrasi Pengklasifikasi .....**Error! Bookmark not defined.**
(2.9) *Mean Absolute Error* .....**Error! Bookmark not defined.**
(2.10) *Mean Squared Error* .....**Error! Bookmark not defined.**
(2.11) *Root Mean Squared Error* .....**Error! Bookmark not defined.**
(2.12) Koefisien Determinasi ($R^2$) .....**Error! Bookmark not defined.**
(2.13) *Mean Absolute Percentage Error* .....**Error! Bookmark not defined.**

# BAB I
# PENDAHULUAN

## 1.1 Latar Belakang

Ketepatan manajemen dalam menyesuaikan harga sewa kamar secara dinamis itu merupakan hal yang diperlukan industri perhotelan modern (Talón-Ballestero dkk., 2022). Untuk memaksimalkan pendapat dan menjaga daya saing pasar kemampuan meramalkan rata-rata harga kamar atau *Average Room Rate* (ARR) secara akurat bisa menjadi salah satu faktor penentu bagi kelangsungan bisnis di industri perhotelan modern. Praktik pengelolaan pendapatan ini mensyaratkan analisis data historis yang sangat komprehensif. Oleh karena itu, pemanfaatan teknik *machine learning* dapat digunakan untuk membaca pola permintaan dan merumuskan strategi harga yang jauh lebih presisi dibandingkan metode konvensional.

Tantangan penetapan harga ini juga dialami oleh PT Genius Utama Indonesia (Genius Hospitality Management) sebagai perusahaan konsultan manajemen yang menaungi berbagai properti di wilayah kepulauan Nusa Tenggara Barat, salah satunya Kaluku Gili Resort. Dalam kegiatan operasionalnya, perusahaan mengelola inventaris kamar yang dipasarkan melalui berbagai saluran distribusi, mulai dari pemesanan langsung (*direct booking*) melalui telepon hingga penggunaan *Online Travel Agent* (OTA) seperti Agoda dan Booking.com. Kompleksitas mulai muncul ketika manajemen harus memproyeksikan harga optimal untuk setiap tipe kamar (*room_id*) di berbagai properti (*property_id*) yang memiliki karakteristik berbeda-beda. Proses analisis secara manual terhadap arus data transaksi dari beragam saluran ini tidak dirasa efisien, mengingat tingginya volume data serta fluktuasi pola musiman pariwisata yang berubah dengan cepat.

Kendala teknis lain yang menghambat bagi operasional perusahaan adalah inkonsistensi kualitas data dari berbagai saluran pemesanan (*booking channels*). Pengamatan awal pada data internal menunjukkan banyaknya rekam data harga sewa kamar yang kosong pada sebagian saluran, yang berpotensi menurunkan performa model prediktif yang akan dibuat. Untuk menyiasati kendala tersebut,

transaksi difokuskan pada saluran Booking.com transaksi yang digunakan difokuskan secara khusus pada data transaksi yang berasal dari saluran Booking.com. Saluran ini dievaluasi memiliki tingkat konsistensi dan kelengkapan data historis yang paling baik, sehingga cukup ideal untuk dijadikan data untuk membantu model memproyeksikan tren harga ke depan.

Berdasarkan pertimbangan-pertimbangan tersebut, penelitian kerja praktik ini disusun untuk merancang dan mengimplementasikan model *forecasting* untuk mengetahui rata-rata harga kamar bulanan menggunakan LightGBM. Sistem prediktif ini dirancang untuk memproyeksikan tren harga secara spesifik untuk masing-masing tipe kamar di setiap properti, sehingga manajemen dapat menggunakan proyeksi ini untuk mengambil langkah yang lebih taktis dan terukur dalam mengeksekusi strategi penetapan harga dinamis.

## 1.2 Rumusan Masalah

Berdasarkan uraian pada latar belakang, persoalan utama yang ingin dipecahkan melalui kerja praktik ini berpusat pada upaya merancang serta mengimplementasikan sebuah model prediksi cerdas. Fokus kajiannya adalah bagaimana algoritma *Light Gradient Boosting Machine* (LightGBM) dapat diterapkan untuk meramalkan rata-rata harga kamar bulanan secara spesifik berdasarkan tipe kamar (*room_id*) dan lokasi properti (*property_id*) di PT Genius Utama Indonesia.

## 1.3 Batasan Masalah

Berikut merupakan batasan masalah dari penelitian kerja praktik agar pembahasan dalam kerja praktik ini tidak terlalu melebar dan tetap berpegang pada tujuan awal, diantaranya:

1. Data yang diproses sepenuhnya mengandalkan rekaman transaksi pemesanan kamar historis milik PT Genius Utama Indonesia dengan rentang periode kedatangan tamu (*arrival date*) mulai 11 Juni 2022 hingga 27 Desember 2026.
2. Sumber pengumpulan data dibatasi secara eksklusif pada satu saluran pemasaran daring saja, yaitu Booking.com. Hal ini diputuskan untuk

menjaga kelengkapan dan konsistensi kualitas data yang kerap menjadi kendala pada saluran lain.

3. Fokus prediksi dilakukan pada level yang cukup detail, yakni dengan mempertimbangkan identitas properti (*property_id*) sekaligus tipe kamar (*room_id*).
4. Variabel target yang menjadi acuan keluaran peramalan adalah rata-rata harga kamar bulanan (*average room rate*).

## 1.4 Tujuan Kerja Praktik

Berdasarkan dengan rumusan masalah yang telah diidentifikasi pelaksanaan kerja praktik ini disusun untuk mencapai beberapa target berikut:

1. Membangun model *forecasting* untuk peramalan harga kamar bulanan yang mampu memproses fitur kompleks, seperti *property_id* dan *room_id* menggunakan algoritma LightGBM.
2. Menghasilkan estimasi harga kamar yang mendekati akurat untuk periode waktu ke depan pada berbagai tipe kamar di bawah pengelolaan PT Genius Utama Indonesia.
3. Mengukur tingkat keandalan dan metrik kesalahan model LightGBM saat berhadapan dengan karakteristik data deret waktu dari riwayat pemesanan di Booking.com.

## 1.5 Manfaat Kerja Praktik

Penelitian ini diharapkan dapat memberi kontribusi yang nyata terhadap instansi tempat studi kasus dilakukan dengan hasil dari kerja praktik ini. Rincian manfaat yang kemungkinan besar bisa diperoleh oleh PT Genius Utama Indonesia meliputi:

1. Menyediakan landasan analitis bagi manajemen PT Genius Utama Indonesia untuk mengambil keputusan penentuan harga dinamis kamar hotel.
2. Membantu tim bagian *Revenue* di perusahaan agar lebih mudah memetakan proyeksi harga rata-rata pada setiap tipe kamar di masa mendatang, yang dapat digunakan untuk menyusun langkah penjualan yang lebih tajam dan

akurat untuk memaksimalkan profitabilitas.

3. Mengurangi beban analisis tren harga secara manual yang memakan waktu, mengingat sistem yang dibangun diharapkan mampu memproses data historis yang bervariasi secara lebih terotomatisasi.

## 1.6 Sistemika Penulisan

Guna memudahkan pembaca dalam menelusuri alur pemikiran dan gambaran utuh dari laporan kerja praktik ini, materi pembahasan disusun berdasarkan sistematika berikut:

**BAB I PENDAHULUAN**

Bab I menguraikan dasar pemikiran mengapa kerja praktik ini dilaksanakan. Uraian dalam Bab I mencakup latar belakang permasalahan, perumusan masalah, batasan penelitian agar penelitian terarah, hingga tujuan dan manfaat yang bisa didapatkan oleh instansi, lalu ditutup dengan sistematika penyusunan laporan.

**BAB II KAJIAN HASIL PENELITIAN DAN LANDASAN TEORI**

Bab II berisi ulasan ringkas mengenai beberapa penelitian terdahulu yang relevan sebagai acuan. Selain itu, bagian ini juga membedah teori-teori dasar yang menopang penyelesaian masalah, mulai dari konsep peramalan data deret waktu hingga prinsip kerja dari algoritma LightGBM yang digunakan dalam pembangunan model.

**BAB III TINJAUAN UMUM INSTANSI**

Bab III membahas mengenai instansi tempat pelaksanaan kerja praktik, yaitu PT Genius Utama Indonesia. Pembahasan pada bab ini meliputi gambaran umum perusahaan, struktur organisasi, serta produk dan jasa yang dihasilkan oleh perusahaan. Penjelasan tersebut bertujuan untuk memberikan pemahaman mengenai profil perusahaan serta kegiatan operasional yang berkaitan dengan topik kerja praktik.

**BAB IV ANALISIS DAN PERANCANGAN**

Bab IV Analisis dan Perancangan menguraikan kerangka metodologis serta pendekatan teknis yang diadopsi untuk menangani data reservasi dari saluran Booking.com. Bagian ini merinci tahapan eksplorasi data, mulai dari penanganan

ketidakkonsistenan *dataset* hingga strategi pra-pemrosesan yang diperlukan untuk membentuk variabel prediktor yang valid. Selain itu, arsitektur model *Light Gradient Boosting Machine* (LightGBM) yang dirancang untuk memprediksi rata-rata harga kamar berdasarkan *property_id* dan *room_id* dipaparkan secara struktural, termasuk skema pembagian data latih dan uji yang digunakan.

**BAB V IMPLEMENTASI DAN PEMBAHASAN**

Bab V Implementasi dan Pembahasan mendemonstrasikan realisasi dari rancangan model *forecasting* yang telah disusun. Fokus utama bagian ini terletak pada proses pelatihan model terhadap data historis serta evaluasi kinerjanya menggunakan metrik pengukuran yang relevan untuk mengukur tingkat akurasi prediksi. Interpretasi terhadap hasil prediksi disajikan secara mendalam untuk melihat kemampuan model dalam menangkap pola fluktuasi harga di berbagai tipe kamar, disertai diskusi mengenai faktor-faktor yang kemungkinan memengaruhi deviasi antara nilai prediksi dan kondisi aktual di lapangan.

**BAB VI PENUTUP**

Bab VI Penutup merangkum intisari temuan yang diperoleh selama pelaksanaan kerja praktik, sekaligus menjawab tujuan penelitian yang telah ditetapkan di awal. Bagian akhir ini turut menyumbangkan sejumlah saran konstruktif, baik untuk pengembangan teknis model prediksi di masa mendatang maupun rekomendasi operasional bagi Genius Hospitality Management dalam memanfaatkan wawasan data yang dihasilkan.

# BAB II
# KAJIAN HASIL PENELITIAN DAN LANDASAN TEORI

## 2.1 Kajian Hasil Penelitian

Penerapan algoritma *machine learning* untuk memprediksi harga dan meramalkan deret waktu telah dieksplorasi pada berbagai sektor industri. Tinjauan terhadap beberapa studi sebelumnya diperlukan untuk memetakan posisi kerja praktik ini, sekaligus melihat potensi metode *Light Gradient Boosting Machine* (LightGBM) saat dihadapkan pada karakteristik data pemesanan kamar hotel.

Pada penelitian Astarani dkk. (2025), dilakukan uji komparasi antara algoritma XGBoost dan LightGBM untuk prediksi penjualan ritel menggunakan dataset Walmart. Hasil pengujian menunjukkan bahwa LightGBM mencatat tingkat kesalahan yang lebih rendah, dengan *Mean Squared Error* (MSE) sebesar 0,0341 dan *Root Mean Squared Error* (RMSE) sebesar 0,1847. Kapasitas LightGBM dalam memproses data berskala besar ini mengindikasikan kecocokannya untuk menangani data transaksi log perhotelan.

Penelitian oleh Rahman dkk. (2025) menggunakan algoritma XGBoost untuk mengestimasi harga kos di Samarinda berdasarkan fitur fasilitas dan lokasi. Meskipun model tersebut menghasilkan *Mean Absolute Percentage Error* (MAPE) sekitar 18%, studi ini memperlihatkan bahwa pemilihan atribut yang relevan sangat menentukan akurasi regresi. Temuan ini mendukung rencana penggunaan atribut pembeda spesifik seperti identitas properti (*property_id*) dan identitas kamar (*room_id*) pada pemodelan yang akan dibangun.

Penelitian Pramita dkk. (2026) telah berhasil memprediksi permintaan hotel menggunakan metode Random Forest Regression. Model yang dikembangkan menunjukkan performa yang sangat baik, dengan nilai R-squared mencapai 99,83% pada data uji, 99,95% pada data latih, serta 88,24% pada prediksi tiga bulan ke depan, disertai nilai MAE, MSE, dan RMSE yang relatif rendah. Hasil tersebut menunjukkan bahwa Random Forest mampu menangkap variasi permintaan hotel secara efektif, terutama pada tipe kamar dengan pola permintaan yang stabil. Berdasarkan analisis mereka, variabel internal hotel seperti tingkat hunian, jumlah

reservasi, dan pembatalan memiliki kontribusi signifikan terhadap akurasi peramalan. Pendekatan berbasis data historis internal ini sejalan dengan arah kerja praktik yang menitikberatkan pada riwayat pemesanan aktual sebagai dasar dalam penentuan harga.

Dari aspek optimasi komputasi, studi Aulady dkk. (2024) mengimplementasikan LightGBM dengan optimasi *Hyper-Parameter Tuning* melalui *GridSearchCV* pada kasus regresi harga mobil bekas. Pendekatan ini terbukti efisien dalam mempercepat waktu pelatihan sekaligus menjaga performa, dibuktikan dengan nilai *R-Square* 0,89 dan MSE 0,034. Karakteristik algoritma yang efisien ini sangat relevan untuk diterapkan saat memproses ragam tipe kamar hotel secara simultan.

Pada sisi pemodelan statistik konvensional, Pujiningrum dan Sulistijanti (2024) meramalkan tingkat hunian hotel bintang lima di Bali menggunakan metode ARIMA dan *Fuzzy Time Series*. Walaupun model ARIMA (1,1,1) mampu mencatat kesalahan di bawah 10% untuk data yang pola musimannya stabil, pendekatan statistik biasa ini kerap menemui hambatan pada data yang fluktuasinya tidak teratur. Senada dengan hal tersebut, studi Herlambang dan Azies (2026) yang membandingkan ARIMA dan Prophet menyimpulkan bahwa metode tradisional umumnya hanya optimal untuk jangka pendek. Pada deret waktu yang lebih dinamis dan kompleks, algoritma non-linier modern lebih diperlukan untuk menangkap variabilitas perubahan tren.

Berdasarkan tinjauan literatur tersebut, pemodelan berbasis *gradient boosting* seperti LightGBM terbukti tangguh untuk kasus regresi pada set data berskala besar. Namun demikian, terlihat adanya celah penelitian (*research gap*) pada implementasinya di industri perhotelan. Mayoritas studi saat ini baru sebatas memprediksi tingkat permintaan umum atau estimasi harga properti secara global. Belum banyak model peramalan yang diarahkan untuk memprediksi *Average Room Rate* (ARR) secara terperinci hingga ke level identitas tipe kamar (*room_id*) per lokasi hotel (*property_id*). Oleh karena itu, kerja praktik ini difokuskan untuk mengisi batasan tersebut. Dengan memanfaatkan data historis tunggal dari saluran Booking.com demi menjaga konsistensi fitur, luaran dari model peramalan ini

ditujukan murni sebagai solusi taktis bagi PT Genius Utama Indonesia. Hasil proyeksi harga model ini akan langsung digunakan oleh manajemen instansi sebagai dasar validasi untuk menetapkan strategi harga dinamis (*dynamic pricing*) yang lebih akurat per tipe kamarnya.

Tabel 2.1 *State of The Art*

| No | Judul Penelitian | Peneliti | Metode | Hasil Pembahasan |
|---|---|---|---|---|
| 1 | Analisis Perbandingan XGBoost dan LightGBM dalam Prediksi Penjualan Ritel Walmart Store Sales | I Gusti Ayu Riyani Astarani dan I Gede Surya Rahayuda | XGBoost, LightGBM | LightGBM menunjukkan performa yang lebih unggul dibandingkan XGBoost dalam menangani data berskala besar, dengan tingkat kesalahan yang lebih rendah, yaitu *Mean Squared Error* (MSE) sebesar 0,0341 dan *Root Mean Squared Error* (RMSE) sebesar 0,1847. |

Tabel 2.1 *State of the Art* (lanjutan)

| No | Judul Penelitian | Peneliti | Metode | Hasil Pembahasan |
|---|---|---|---|---|
| 2 | Penerapan Algoritma XGBoost Dalam Prediksi Harga Sewa Kos Di Kota Samarinda | Amalia Rahman, Amelia Yusnita, dan Hanifah Ekawati | XGBoost | Model menghasilkan nilai *Mean Absolute Percentage Error* (MAPE) sekitar 18%. Studi ini menyimpulkan bahwa pemilihan atribut fisik dan proper yang relevan sangat krusial dalam meningkatkan akurasi regresi harga properti. |

Tabel 2.1 *State of the Art* (lanjutan)

| No | Judul Penelitian | Peneliti | Metode | Hasil Pembahasan |
|---|---|---|---|---|
| 3 | Forecasting Hotel Demand with Time Series Prediction Model Using Random Forest Regression | Dewa Ayu Kadek Pramita, Ni Wayan Sumartini Saraswati, I Putu Dedy Sandana, Dewa Ayu Putu Rasmika Dewi, dan Ni Kadek Bumi Krismentari | Random Forest Regression | Model mencapai akurasi sangat tinggi dengan nilai *R-squared* 99,83% pada data uji dan 88,24% untuk prediksi tiga bulan ke depan. Variabel internal seperti tingkat hunian dan jumlah reservasi terbukti memiliki kontribusi signifikan terhadap akurasi peramalan. |

Tabel 2.1 *State of the Art* (lanjutan)

| No | Judul Penelitian | Peneliti | Metode | Hasil Pembahasan |
|---|---|---|---|---|
| 4 | Improve Metode LightGBM untuk Prediksi Harga Mobil Bekas Menggunakan Hyper-Parameter Tuning | Moch. Aqil Aulady, Ahmad Hudawi AS., dan Zainal Arifin | LightGBM, Hyper-Parameter Tuning (GridSearchCV) | Implementasi *Hyper-Parameter Tuning* terbukti efisien dalam mempercepat waktu pelatihan sekaligus menjaga performa model, yang dibuktikan dengan pencapaian nilai *R-Square* sebesar 0,89 dan MSE sebesar 0,034. |

Tabel 2.1 *State of the Art* (lanjutan)

| No | Judul Penelitian | Peneliti | Metode | Hasil Pembahasan |
|---|---|---|---|---|
| 5 | Peramalan Tingkat Penghunian Kamar Hotel Bintang Lima Provinsi Bali Menggunakan Metode ARIMA dan Fuzzy Time Series Lee | Rahayu Dwiastuti Pujiningrum dan Wellie Sulistijanti | ARIMA, Fuzzy Time Series Lee | Model ARIMA (1,1,1) mampu mencatat kesalahan di bawah 10% pada data dengan pola musiman stabil. Namun, metode statistik konvensional ini ditemukan kurang optimal dalam menangani data dengan fluktuasi yang tidak teratur. |

Tabel 2.1 *State of the Art* (lanjutan)

| No | Judul Penelitian | Peneliti | Metode | Hasil Pembahasan |
|---|---|---|---|---|
| 6 | Time Series Forecasting Saham PT Astra Menggunakan Algoritma Autoregressive Integrated Moving Average dan Prophet | Naufal Malik Herlambang dan Harun Al Azies | ARIMA, Prophet | Hasil komparasi menunjukkan bahwa metode tradisional umumnya optimal untuk jangka pendek. Pada deret waktu yang dinamis dan kompleks, algoritma non-linier modern lebih diperlukan untuk menangkap variabilitas perubahan tren. |

## 2.2 Landasan Teori

### 2.2.1 Manajemen Pendapatan Hotel

Praktik manajemen pendapatan di sektor perhotelan biasanya berpusat pada melaraskan antara ketersediaan aset dengan perubahan permintaan konsumen. Pendekatan ini tidak sekadar tentang mengatur tarif dasar penginapan, melainkan lebih jauh melibatkan strategi penetapan harga dinamis (*dynamic pricing*) yang dapat merespons berbagai faktor.

Beberapa faktor yang biasanya memengaruhi penyesuaian harga ini meliputi tingkat hunian saat ini (*occupancy*), tren pemesanan dari saluran daring, hingga siklus musiman pariwisata seperti periode sepi maupun puncak liburan

(Santos dkk., 2024). Karakteristik kamar hotel sebagai produk jasa yang tidak dapat disimpan atau dijual kembali pada hari berikutnya menyebabkan kegagalan dalam menyewakan kamar pada suatu malam berpotensi memicu hilangnya pemasukan secara permanen bagi manajemen. Oleh karena itu, pengelola properti bisa mendapatkan manfaat dari analisis riwayat data transaksi untuk memproyeksikan harga yang paling ideal. Pihak manajemen kemudian dapat menyasar segmen tamu yang tepat pada waktu yang paling menguntungkan proyeksi harga tersebut.

Dalam industri perhotelan bisa menggunakan indikator rata-rata harga kamar atau juga dikenal *Average Room Rate* (ARR) untuk mengevaluasi seberapa efektif strategi penetapan harga dinamis. Sebagian literatur perhotelan juga sering menyebut metrik yang sama dengan istilah *Average Daily Rate* (ADR). Indikator ini merepresentasikan rata-rata uang yang didapat dari setiap unit kamar yang berhasil disewakan dalam kurun waktu tertentu. Dalam kajian prediksi berbasis deret waktu, nilai ARR kerap kali diposisikan sebagai variabel target peramalan utama karena kemampuannya memotret realitas harga transaksi di lsapangan. Secara matematis, besaran nilai ARR diperoleh melalui pembagian antara total pendapatan dari penjualan kamar dengan jumlah unit yang laku terjual (Darma dan Wirawati, 2019).

Bentuk perhitungan tersebut dijabarkan lebih lanjut pada Persamaan 2.1.

$$ARR = \frac{\text{Total Pendapatan Kamar}}{\text{Total Kamar Terjual}} \tag{2.1}$$

Pemahaman mengenai fluktuasi ARR ini nantinya akan menjadi landasan penting dalam mengoptimalkan algoritma peramalan untuk memproyeksikan harga sewa yang paling kompetitif pada masa mendatang.

**Contoh 2.1**

Berdasarkan data transaksi kamar pada bulan Juni 2022, diketahui:

1. Total Pendapatan Kamar $= 750.776.200.000$ Rupiah
2. Total Kamar Terjual $= 793$ kamar

   *Average Room Rate* (ARR) dihitung dengan membagi total pendapatan

kamar dengan jumlah kamar yang terjual, sesuai dengan Persamaan 2.1:

$$\text{ARR} = \frac{\text{Total Pendapatan Kamar}}{\text{Total Kamar Terjual}}$$

Langkah perhitungan:

1. Menentukan nilai total pendapatan dan kamar terjual
   Dari data transaksi bulan Juni 2022:
   $$\text{Total Pendapatan Kamar} = \text{Rp } 750.776.200.000$$
   $$\text{Total Kamar Terjual} = 793 \text{ kamar}$$
2. Menghitung ARR
   Masukkan nilai ke dalam Persamaan 2.1:
   $$\text{ARR} = \frac{750.776.200.000}{793}$$
   $$\text{ARR} \approx \text{Rp } 946.754 \text{ per kamar per malam}$$
3. Interpretasi hasil
   Nilai ARR sebesar Rp 946.754 menunjukkan bahwa rata-rata pendapatan yang diperoleh dari setiap kamar yang terjual pada bulan Juni 2022 adalah sekitar Rp 946.754 per malam.

Perhitungan ini menunjukkan bagaimana ARR dapat digunakan sebagai indikator performa pendapatan kamar setiap bulan. Metode ini juga dapat diterapkan pada bulan-bulan berikutnya dengan cara yang sama, hanya mengganti nilai total pendapatan dan kamar terjual sesuai data masing-masing bulan.

### *2.2.2 Time Series Forecasting*

Deret waktu (*time series*) merupakan sekumpulan hasil pengamatan terhadap suatu variabel yang dicatat secara berurutan berdasarkan waktu (Montgomery dan Jennings, 2024). Proses peramalan pada data semacam ini pada intinya berupaya mengekstrak pola dari riwayat masa lalu untuk memproyeksikan angka di masa depan (Truong dan Novák, 2023). Khususnya pada peramalan kuantitatif jangka pendek hingga menengah, keberhasilannya sangat bergantung pada seberapa baik pola historis tersebut diidentifikasi dan diekstrapolasi. Bagi

manajemen perhotelan, informasi proyeksi ini sering kali menjadi acuan krusial dalam menyusun perencanaan pendapatan operasional.

Secara umum, struktur data deret waktu diasumsikan terbentuk dari empat komponen utama. Keempat komponen tersebut meliputi tren yang memperlihatkan arah pergerakan jangka panjang, siklus, pola musiman yang merepresentasikan naik-turunnya data pada rentang waktu berulang, serta gangguan acak (*noise*) yang sifatnya tidak terstruktur (Truong dan Novák, 2023). Sektor perhotelan menunjukkan pola musiman dan tren yang jelas, di mana permintaan serta harga kamar mengalami fluktuasi yang cenderung mengikuti pola tertentu seperti libur panjang atau akhir pekan. Dekomposisi deret waktu secara matematis dapat dirumuskan pada Persamaan 2.2 (Truong dan Novák, 2023).

$$X(t) = Tr(t) + C(t) + S(t) + R(t) \tag{2.2}$$

Keterangan:

$X(t)$ = Nilai deret waktu pada saat $t$
$Tr(t)$ = Komponen tren (*trend*)
$C(t)$ = Komponen siklus (*cycle*)
$S(t)$ = Komponen musiman (*seasonality*)
$R(t)$ = Komponen acak (*random noise*)

Penerapan algoritma *machine learning* pada deret waktu tidak bisa dilakukan begitu saja. Data mentah perlu melalui tahap transformasi agar model dapat mengenali sifat autokorelasi, yakni kecenderungan di mana suatu nilai pada waktu tertentu dipengaruhi oleh rentetan nilai sebelumnya (Montgomery dan Jennings, 2024). Agar data ini dapat diproses dalam kerangka supervised learning, penerapan rekayasa fitur (*feature engineering*) mutlak diperlukan. Beberapa teknik yang lazim dipakai adalah pembuatan variabel *lag*, yang menggeser observasi masa lalu menjadi fitur masukan baru, serta rata-rata bergerak (*rolling window*) guna meratakan fluktuasi data jangka pendek.

Melalui pendekatan rekayasa fitur tersebut, algoritma prediktif dapat mempelajari riwayat pergerakan harga kamar secara lebih spesifik. Model pada

akhirnya mampu menangkap pola historis unik dari tiap identitas properti (*property_id*) maupun tipe kamar *(room_id*), sehingga proyeksi rata-rata harga bulanan yang dihasilkan bisa lebih presisi dan dapat diandalkan untuk strategi penetapan harga dinamis.

**Contoh 2.2**

Sebagai ilustrasi perhitungan dekomposisi deret waktu, digunakan data *Average Room Rate* (ARR) bulanan dari Juni 2022 hingga Mei 2023 sebagaimana disajikan pada Tabel 2.3 sebagai berikut:

Tabel 2.2 Data ARR

| Tahun | Bulan | Total Pendapatan Kamar (Rp) | Total Kamar Terjual (kamar) | ARR (Rp) |
|---|---|---|---|---|
| 2022 | Juni | 750.776.200.000 | 793 | 946.754 |
| 2022 | Juli | 2.193.926.000.000 | 1.716 | 1.278.512 |
| 2022 | Agustus | 8.304.128.000.000 | 6.032 | 1.376.679 |
| 2022 | September | 7.042.278.000.000 | 4.875 | 1.444.570 |
| 2022 | Oktober | 6.234.973.000.000 | 4.563 | 1.366.420 |
| 2022 | November | 4.787.362.000.000 | 3.601 | 1.329.453 |
| 2022 | Desember | 7.011.261.000.000 | 3.952 | 1.774.105 |
| 2023 | Januari | 6.333.431.000.000 | 3.627 | 1.746.190 |
| 2023 | Februari | 5.528.212.000.000 | 3.055 | 1.809.562 |
| 2023 | Maret | 7.399.731.000.000 | 4.485 | 1.649.884 |
| 2023 | April | 9.900.916.000.000 | 5.694 | 1.738.833 |
| 2023 | Mei | 15.789.140.000.000 | 10.218 | 1.545.228 |

Setiap nilai ARR pada bulan ke-*t* dinotasikan sebagai *X(t)*, sehingga rentang observasi yang tersedia adalah *t = 1, 2, ..., 12*, berturut-turut mewakili periode Juni 2022 hingga Mei 2023. Mengacu pada Persamaan 2.2, nilai deret waktu pada setiap bulan dapat diurai menjadi:

$$X(t) = Tr(t) + C(t) + S(t) + R(t)$$

Dalam konteks data ini, komponen siklus *C(t)* diasumsikan bersifat konstan

dalam jangka pendek sehingga tidak dapat diidentifikasi secara terpisah dari komponen musiman pada cakupan data 12 bulan. Oleh karena itu, pada ilustrasi ini *C(t)* dimasukkan ke dalam residu gabungan bersama *S(t)* dan *R(t)*. Fokus perhitungan diarahkan pada estimasi komponen tren *Tr(t)* menggunakan metode regresi linear kuadrat terkecil (*Ordinary Least Squares*), kemudian menghitung deviasi tiap observasi terhadap garis tren tersebut sebagai estimasi gabungan *S(t)* + *C(t)* + *R(t)*.

**Langkah 1: Penomoran Periode dan Statistik Dasar**

Setiap bulan observasi diberi indeks *t* secara berurutan mulai dari *t = 1* (Juni 2022) hingga *t = 12* (Mei 2023). Berdasarkan data pada Tabel 2.3, diperoleh statistik ringkas sebagai berikut:

$$n = 12$$

$$\Sigma t = 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 + 11 + 12 = 78$$

$$\begin{aligned}\Sigma x = {} & 946.754 + 1.278.512 + 1.376.679 + 1.444.570 + 1.366.420 \\ & + 1.329.453 + 1.774.105 + 1.746.190 + 1.809.562 \\ & + 1.649.884 + 1.738.833 + 1.545.228 \\ = {} & 18.006.190\end{aligned}$$

$$\Sigma t^2 = 1^2 + 2^2 + 3^2 + 4^2 + 5^2 + 6^2 + 7^2 + 8^2 + 9^2 + 10^2 + 11^2 + 12^2 = 650$$

$$\begin{aligned}\Sigma t \cdot x = {} & (1 \times 946.754) + (2 \times 1.278.512) + (3 \times 1.376.679) \\ & + (4 \times 1.444.570) + (5 \times 1.366.420) + (6 \times 1.329.453) \\ & + (7 \times 1.774.105) + (8 \times 1.746.190) + (9 \times 1.809.562) \\ & + (10 \times 1.649.884) + (11 \times 1.738.833) + (12 \times 1.545.228) \\ = {} & 125.063.965\end{aligned}$$

$$\bar{t} = 78/12 = 6{,}5$$

$$\bar{x} = 18.006.190/12 = 1.500.515{,}83$$

**Langkah 2: Estimasi Komponen Tren dengan Regresi Linear**

Komponen tren $Tr(t)$ diestimasi menggunakan persamaan garis lurus $Tr(t) = \beta^0 + \beta^1 \cdot t$. Koefisien kemiringan $\beta^1$ dan intersep $\beta^0$ diperoleh melalui rumus OLS berikut:

$$\beta^1 = (\Sigma t \cdot x - n \cdot \bar{t} \cdot \bar{x})/(\Sigma t^2 - n \cdot \bar{t}^2)$$
$$\beta^1 = (125.063.965 - 12 \times 6{,}5 \times 1.500.515{,}83)/(650 - 12 \times 6{,}5^2)$$
$$\beta^1 = (125.063.965 - 117.040.235)/(650 - 507)$$
$$\beta^1 = 8.023.730/143 \approx 56.110$$
$$\beta^0 = \bar{x} - \beta^1 \cdot \bar{t} = 1.500.515{,}83 - 56.110 \times 6{,}5 = 1.135.800{,}83$$

Dengan demikian, persamaan tren yang diperoleh adalah:

$$Tr(t) = 1.135.800{,}83 + 56.110 \times t$$

Nilai $\beta_1 = 56.110$ mengindikasikan bahwa secara rata-rata, ARR mengalami kenaikan sebesar Rp 56.110 setiap bulannya sepanjang periode observasi. Hal ini mencerminkan adanya tren jangka panjang yang cenderung meningkat pada tarif kamar.

**Langkah 3: Perhitungan Nilai Tren dan Deviasi untuk Seluruh Periode**

Tabel berikut menyajikan nilai tren *Tr(t)* yang diperoleh dari persamaan regresi, serta deviasi *X(t) − Tr(t)* yang merepresentasikan gabungan komponen musiman, siklus, dan acak untuk setiap periode:

Tabel 2.3 Hasil Perhitungan Tren dan Deviasi

| t | Periode | $X(t)$ $- ARR(Rp)$ | $Tr(t)$ $= 1.135.800{,}83$ $+ 56.110 \cdot t$ | $S(t) + C(t)$ $+ R(t)$ $= X(t)$ $- Tr(t)$ | Keterangan |
|---|---|---|---|---|---|
| 1 | 2022-06 | 946.754 | 1.191.911 | −245.157 | Di bawah tren (low season awal) |
| 2 | 2022-07 | 1.278.512 | 1.248.021 | +30.491 | Sedikit di atas tren |
| 3 | 2022-08 | 1.376.679 | 1.304.131 | +72.548 | Di atas tren (mulai high season) |
| 4 | 2022-09 | 1.444.570 | 1.360.241 | +84.329 | Di atas tren |
| 5 | 2022-10 | 1.366.420 | 1.416.351 | −49.931 | Sedikit di bawah tren |
| 6 | 2022-11 | 1.329.453 | 1.472.461 | −143.008 | Di bawah tren (low season) |
| 7 | 2022-12 | 1.774.105 | 1.528.571 | +245.534 | Jauh di atas tren (peak season) |
| 8 | 2023-01 | 1.746.190 | 1.584.681 | +161.509 | Di atas tren |
| 9 | 2023-02 | 1.809.562 | 1.640.791 | +168.771 | Di atas tren |
| 10 | 2023-03 | 1.649.884 | 1.696.901 | −47.017 | Mendekati tren |
| 11 | 2023-04 | 1.738.833 | 1.753.011 | −14.178 | Mendekati tren |
| 12 | 2023-05 | 1.545.228 | 1.809.121 | −263.893 | Di bawah tren |

**Langkah 4: Substitusi Nilai ke Persamaan Dekomposisi**

Sebagai ilustrasi konkret, berikut disajikan substitusi nilai aktual ke dalam

Persamaan 2.2 untuk dua periode representatif, yakni Desember 2022 (*t = 7*) yang mencerminkan puncak musim ramai (*peak season*), dan Mei 2023 (*t = 12*) yang berada di bawah garis tren.

**a) Periode Desember 2022 (t = 7):**

Nilai ARR aktual pada bulan Desember 2022 adalah *X(7) = 1.774.105*. Nilai tren pada periode tersebut diperoleh dengan mensubstitusi *t = 7* ke dalam persamaan regresi:

$$Tr(7) = 1.135.800{,}83 + 56.110 \times 7 = 1.135.800{,}83 + 392.770$$
$$= 1.528.570{,}83 \approx 1.528.571$$

Dengan demikian, deviasi gabungan komponen musiman, siklus, dan acak adalah:

$$S(7) + C(7) + R(7) = X(7) - Tr(7) = 1.774.105 - 1.528.571 = +245.534$$

Sehingga persamaan dekomposisi untuk periode ini dapat dituliskan:

$$X(7) = Tr(7) + S(7) + C(7) + R(7)$$
$$1.774.105 \ = \ 1.528.571 \ + \ 245.534$$

Nilai deviasi positif sebesar Rp 245.534 mengindikasikan bahwa ARR pada Desember 2022 berada jauh di atas garis tren jangka panjang. Kondisi ini kemungkinan besar dipengaruhi oleh lonjakan permintaan pada periode liburan akhir tahun (*peak season*), yang tercermin sebagai komponen musiman positif pada periode tersebut.

**b) Periode Mei 2023 (t = 12):**

Nilai ARR aktual pada bulan Mei 2023 adalah *X(12) = 1.545.228*. Nilai tren diperoleh melalui substitusi *t = 12*:

$$Tr(12) = 1.135.800{,}83 + 56.110 \times 12 = 1.135.800{,}83 + 673.320$$
$$= 1.809.120{,}83 \approx 1.809.121$$

Deviasi gabungan komponen non-tren:

$$S(12) + C(12) + R(12) = X(12) - Tr(12) = 1.545.228 - 1.809.121$$
$$= -263.893$$

Persamaan dekomposisi untuk periode ini menjadi:

$$X(12) = Tr(12) + S(12) + C(12) + R(12)$$
$$1.545.228 = 1.809.121 + (-263.893)$$

Deviasi negatif sebesar −Rp 263.893 menunjukkan bahwa meskipun garis tren linear masih memprediksi kenaikan harga pada *t = 12*, nilai ARR aktual justru tercatat lebih rendah dari proyeksi tren. Hal ini dapat mengindikasikan pengaruh musiman negatif atau komponen acak yang menekan harga pada bulan tersebut.

**Interpretasi Hasil Dekomposisi**

Berdasarkan hasil perhitungan pada seluruh dua belas periode observasi, tren jangka panjang ARR menunjukkan peningkatan sebesar Rp 56.110 per bulan. Pola ini konsisten dengan pertumbuhan permintaan akomodasi di wilayah Gili Islands selama periode pengamatan. Adapun deviasi positif yang cukup besar terlihat pada bulan-bulan akhir tahun (November–Februari), yang bersesuaian dengan periode libur panjang dan *high season* pariwisata. Sebaliknya, deviasi negatif dominan muncul pada periode peralihan musim seperti November dan Mei. Pola berulang ini mengindikasikan keberadaan komponen musiman *S(t)* yang signifikan dalam data ARR, meskipun estimasi numerik kuantitatif komponen tersebut secara terpisah memerlukan cakupan data multitahunan yang lebih panjang.

Kerangka dekomposisi ini menjadi landasan penting dalam perancangan fitur-fitur masukan pada model LightGBM. Pola tren yang teridentifikasi melalui analisis ini akan direpresentasikan ke dalam fitur lag dan *rolling window* yang memungkinkan algoritma mempelajari autokorelasi temporal dalam data ARR secara lebih sistematis pada tahap rekayasa fitur (*feature engineering*).

### 2.2.3 Decision Tree

Pohon keputusan (*decision tree*) merupakan salah satu model *supervised machine learning* yang dapat difungsikan untuk memecahkan persoalan regresi maupun klasifikasi. Karena target prediksi dalam penentuan tarif kamar berupa nilai kontinu, pembahasan difokuskan pada pendekatan pohon regresi (*regression trees*). Model regresi ini bekerja secara spesifik dengan mempartisi ruang prediktor atau

sekumpulan fitur input ke dalam beberapa wilayah yang terpisah dan tidak saling tumpang tindih.

Secara konseptual, pembentukan partisi wilayah $R_1, R_2, \dots, R_J$ bertujuan untuk menemukan batas yang meminimalkan *Residual Sum of Squares* (RSS). Mengacu pada rumusan (James dkk., 2023), kriteria kesalahan ini kemungkinan besar dievaluasi pada setiap daun (*leaf*) di struktur pohon melalui Persamaan (2.3):

$$\sum_{j=1}^{J} \sum_{i \in R_j} \left( y_i - \hat{y}_{R_j} \right)^2 \tag{2.3}$$

Di mana $\hat{y}_{R_j}$ adalah rata-rata nilai observasi data latih yang jatuh ke dalam wilayah ke-$j$. Melakukan komputasi komprehensif untuk mencari bentuk wilayah paling optimal secara menyeluruh diduga sangat mustahil dilakukan karena keterbatasan sumber daya. Sebagai jalan tengah, algoritma pohon keputusan lazimnya menerapkan mekanisme rakus (*greedy*) dari atas ke bawah yang dikenal sebagai *recursive binary splitting* .

Mekanisme ini menyeleksi sebuah variabel prediktor $X_j$ beserta titik potong (*cutpoint*) $s$ sedemikian rupa, sehingga ruang observasi yang terbelah mampu menyumbangkan penurunan RSS paling maksimal. Dua area belahan tersebut dapat diekspresikan secara matematis pada Persamaan (2.5):

$$R_1(j, s) = \left\{ X \mid X_j < s \right\} \tag{2.4}$$

$$R_2(j, s) = \{ X \mid X_j \geq s \} \tag{2.5}$$

Pada setiap langkah percabangan, variabel $j$ dan batas $s$ terus dicari untuk menekan nilai kerugian gabungan. Formulasi optimasi pada pemisahan simpul (*node*) ini ditunjukkan oleh Persamaan (2.6) berikut:

$$\mathrm{RSS}(j, s) = \sum_{i: x_i \in R_1(j,s)} \left( y_i - \hat{y}_{R_1} \right)^2 + \sum_{i: x_i \in R_2(j,s)} \left( y_i - \hat{y}_{R_2} \right)^2 \tag{2.6}$$

Berdasarkan perumusan di atas, $\hat{y}_{R_1} dan\ \hat{y}_{R_2}$ berturut-turut merupakan nilai rata-rata variabel target pada masing-masing kelompok $R_1(j,s)$ dan $R_2(j,s)$. Langkah pemisahan biner ini akan terus berulang hingga menemui kriteria penghentian tertentu, misalnya batas minimum jumlah data dalam sebuah daun.

Meskipun proses pelatihannya relatif mudah dipahami, struktur pohon tunggal yang dibiarkan tumbuh terlalu dalam dapat memicu terjadinya *overfitting*. Untuk menyiasati risiko tersebut, strategi *cost complexity pruning* kerap kali disertakan untuk mengidentifikasi bentuk sub-pohon yang memiliki performa paling seimbang (James dkk., 2023). Fungsi kerugian yang dikenakan penalti parameter kompleksitas (α) diuraikan pada Persamaan (2.7):

$$C_\alpha(T) = \sum_{m=1}^{|T|} \sum_{i:x_i \in R_m} \left(y_i - \hat{y}_{R_m}\right)^2 + \alpha|T| \tag{2.7}$$

Penambahan variabel $|T|$ yang merepresentasikan total daun bertujuan memberikan batasan penalti, sehingga peningkatan nilai $\alpha$ akan memaksa model memangkas cabang yang kurang esensial. Di sisi lain, *decision tree* tunggal mungkin masih menyimpan kelemahan dari aspek varians prediksi. Oleh karena alasan inilah, fondasi pemodelan pohon tersebut sering dimanfaatkan sebagai pembelajar dasar (*base learner*) yang dikombinasikan secara bertahap dalam algoritma *gradient boosting*, dengan tujuan akhir merangkai sebuah fungsi prediktor yang jauh lebih presisi dan stabil (Gonçalves dan Carvalho, 2025).

**Contoh 2.3**

Tabel 2.4 Dataset Peramalan

| *Month* | *lag_1* | *price* |
|---|---|---|
| 5 | 406.650 | 833.627,5 |
| 6 | 833.627,5 | 800.829,7 |
| 7 | 800.829,7 | 1.048.560 |

Diketahui bahwa dataset yang digunakan untuk proses peramalan (*forecasting*) adalah sebagaimana telah disajikan sebelumnya.
Rumus yang digunakan untuk menghitung nilai split pada *decision tree* adalah sebagai berikut:

$$\mathrm{RSS}(j,s) = \sum_{i:x_i \in R_1(j,s)} \left(y_i - \hat{y}_{R_1}\right)^2 + \sum_{i:x_i \in R_2(j,s)} \left(y_i - \hat{y}_{R_2}\right)^2$$

Nilai RSS tersebut dihitung untuk setiap fitur (kolom) dan untuk setiap kandidat titik pemisah $s$, yang dsiperoleh dari nilai tengah (midpoint) antara dua observasi berurutan pada kolom yang sama setelah diurutkan.
Berikut merupakan tahapan perhitungan untuk menentukan split optimal berdasarkan nilai *Residual Sum of Squares* (RSS).

**1. *Node* pada kedalaman 1:**

Untuk j=1:

$$s = \frac{5+6}{2} = 5{,}5$$

$$R_1(1, 5.5) = \{x_1\}$$

$$R_2(1, 5.5) = \{x_2, x_3\}$$

$$\begin{aligned} RSS(1, 5.5) = {} & (833.627{,}46 - 833.627{,}4648)^2 \\ & + (800.829{,}74 - 924.694{,}6689)^2 \\ & + (1.048.559{,}60 - 924.694{,}6689)^2 = 30.685.041.372{,}63 \end{aligned}$$

$$s = \frac{6+7}{2} = 6.5$$

$$R_1(1{,}6.5) = \{x_1, x_2\}$$

$$R_2(1{,}6.5) = \{x_3\}$$

$$RSS(1{,}6.5) = (833.627{,}46 - 817.228{,}6023)^2 + (800.829{,}74 - 817.228{,}6023)^2 + (1.048.559{,}60 - 1.048.559{,}5981)^2 = 537.845.384{,}64$$

Untuk j=2:

$$s = \frac{406.650 + 800.830}{2} = 603.740$$

$$R_1(2, 603.740) = \{x_1\}$$

$$R_2(2, 603.740) = \{x_2, x_3\}$$

$$RSS(2, 603.740) = (833.627{,}46 - 833.627{,}4648)^2 + (800.829{,}74 - 924.694{,}6689)^2 + (1.048.559{,}60 - 924.694{,}6689)^2 = 30.685.041.372{,}63$$

$$s = \frac{800.830 + 833.627}{2} = 817.229$$

$$R_1(2, 817.229) = \{x_1, x_3\}$$

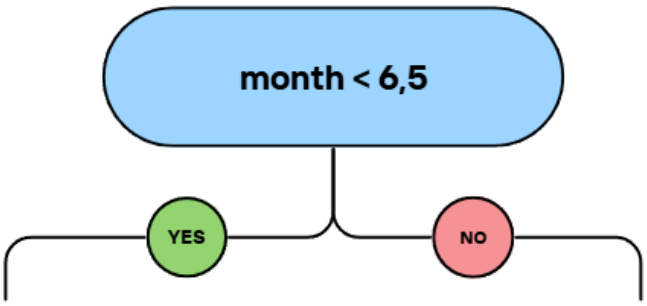
$$R_2(2, 817.229) = \{x_2\}$$

$$RSS(2, 817.229) = (833.627{,}46 - 941.093{,}5315)^2 + (1.048.559{,}60 - 941.093{,}5315)^2 + (800.829{,}74 - 800.829{,}7397)^2 = 23.097.910.971{,}49$$

Jumlah kuadrat selisih (RSS) untuk *node* ini dihitung sebagai:

$(833.627{,}46\text{-}817.228{,}6023)^2$ + $(800.829.74\text{-}817.228{,}6023)^2$ + $(1.048.559.60\text{-}1.048.559{,}5981)^2 = 537.845.384{,}64$

Berdasarkan perhitungan ini, *split* yang dipilih menggunakan fitur *month* dengan *threshold* 6,5, menghasilkan RSS sebesar 537.845.384,64.




Gambar 2.1 *Decision Tree Depth* 1

**2. *Node* pada kedalaman 2:**

Selanjutnya, pohon dilanjutkan pada *node* kiri dengan data sebagai berikut:

Tabel 2.5 Dataset *Node* Kiri Kedalam 2

| *month* | *lag_1* | *price* |
|---|---|---|
| 5 | 406.650 | 833.627,5 |
| 6 | 833.627,5 | 800.829,7 |

Untuk j=1:

$$s = \frac{5+6}{2} = 5{,}5$$

$$R_1(1{,}5.5) = \{x_1\}$$

$$R_2(1{,}5.5) = \{x_2\}$$

$$RSS(1, 5.5) = (833.627{,}46 - 833.627{,}4648)^2 + (800.829{,}74 - 800.829{,}7397)^2 = 0$$

Untuk j=2:

$$s = \frac{406.650 + 833.627}{2} = 620.139$$

$$R_1(2, 620.139) = \{x_1\}$$

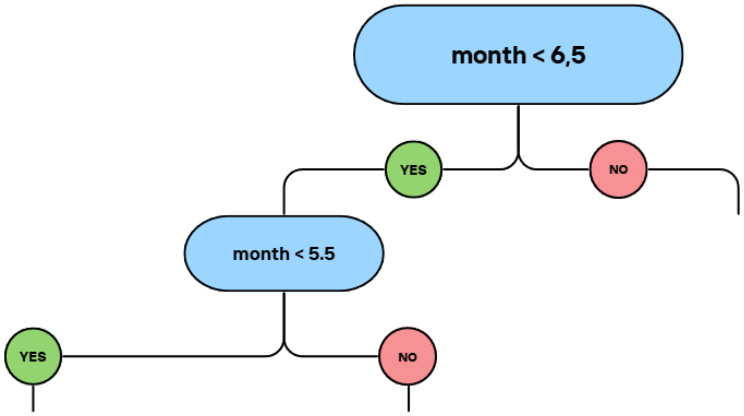
$$R_2(2, 620.139) = \{x_2\}$$

$$RSS(2, 620.139) = (833.627{,}46 - 833.627{,}4648)^2 + (800.829{,}74 - 800.829{,}7397)^2 = 0$$

Jumlah kuadrat selisih (RSS) untuk node ini dihitung sebagai:

$$(833.627{,}46 - 833.627{,}4648)^2 + (800.829{,}74 - 800.829{,}7397)^2 = 0$$

Berdasarkan perhitungan ini, *split* yang dipilih menggunakan fitur *month* dengan *threshold* 5,5, menghasilkan RSS sebesar 0.



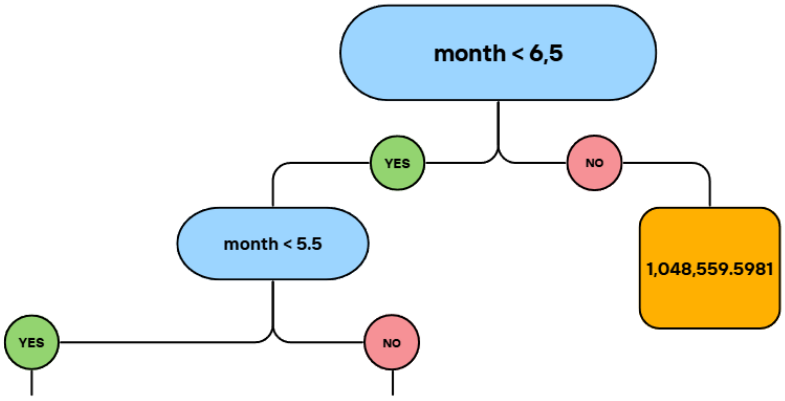

Gambar 2.2 *Decision Tree Node* Kiri *Depth 2*

Selanjutnya, pohon dilanjutkan pada *node* kanan dengan data sebagai berikut:

Tabel 2.6 Dataset *Node* Kanan Kedalam 2

| *month* | *lag_1* | *price* |
|---|---|---|
| 7 | 800.829,739726 | 1.048.559,598131 |

Karena *node* ini hanya memiliki satu sampel, *node* ini dijadikan *leaf* dengan nilai prediksi = 1.048.559,5981.




Gambar 2.3 *Decision Tree Node* Kanan *Depth* 2

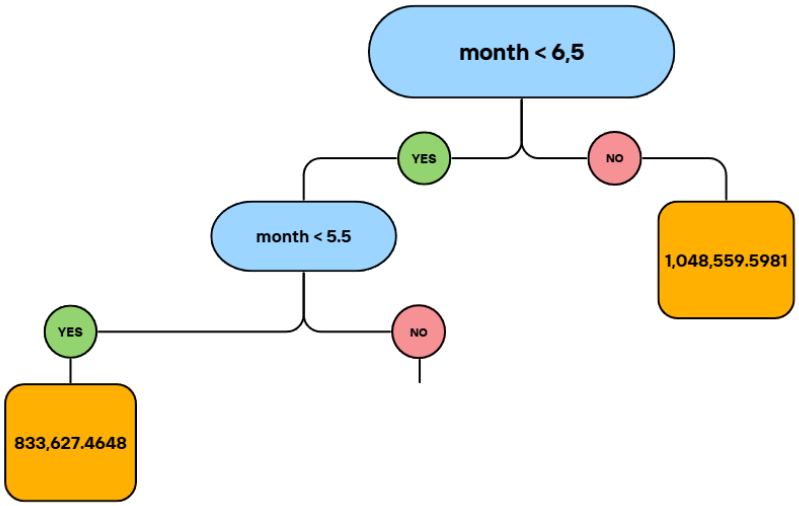
**3. *Node* pada kedalaman 3:**

Selanjutnya, pohon dilanjutkan pada *node* kiri dengan data sebagai berikut:

Tabel 2.7 Dataset *Node* Kiri Kedalam 3

| *month* | *lag_1* | *price* |
|---|---|---|
| 5 | 406.650 | 833.627,464789 |

Karena *node* ini hanya memiliki satu sampel, maka *node* ini akan dijadikan *leaf* dengan nilai prediksi 833.627,4648.

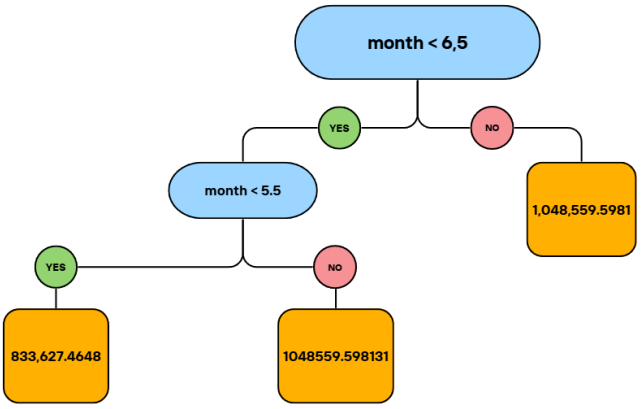



Gambar 2. 4 *Decision Tree Node* Kiri *Depth* 3

Selanjutnya, pohon dilanjutkan pada *node* kanan dengan data sebagai berikut:

Tabel 2. 8 Dataset *Node* Kanan Kedalam 3

| *month* | *lag_1* | *price* |
|---|---|---|
| 7 | 800.829,739726 | 1.048.559,598131 |




Gambar 2.5 *Decision Tree Node* Kanan *Depth* 3

***Pruning*:**

Tabel 2.9 Data *Node* Kiri dan Kanan Sebelum *Pruning*

| Leaf | Nilai Aktual $y_i$ |
|---|---|
| Leaf kiri | 833.627,4648 |
| Leaf kanan | 800.829,7397 |

Rumus *Cost-Complexity Pruning* (CCP):

$$C_\alpha(T) = \sum_{m=1}^{|T|} \sum_{i:x_i \in R_m} \left(y_i - \hat{y}_{R_m}\right)^2 + \alpha|T|$$

dimana:

$y_i$ = nilai aktual pada leaf $m$

$\hat{y}_{R_m}$ = rata-rata nilai $y$ pada leaf $m$

$\alpha$ = parameter penalti kompleksitas (*ccp_alpha*)

$|T|$ = jumlah *leaf* pada pohon

**Tahap 1: Perhitungan Rata-Rata Setiap *Leaf***

Leaf kiri:

$$\hat{y}_{\text{kiri}} = 833.627{,}4648$$

Leaf kanan:

$$\hat{y}_{\text{kanan}} = 800.829{,}7397$$

**Tahap 2: Perhitungan Kuadrat Selisih Tiap Observasi**

Leaf kiri:

$$(833.627{,}46 - 833.627{,}4648)^2 = 0$$

Leaf kanan:

$$(800.829{,}74 - 800.829{,}7397)^2 = 0$$

**Tahap 3: Perhitungan Total *Loss* Sebelum *Pruning***

Total *Loss* sebelum *prune* $= \sum_{\text{Leaf kiri}} (y_i - \hat{y}_{R_m})^2 + \sum_{\text{Leaf kanan}} (y_i - \hat{y}_{R_m})^2 + \alpha \mid T \mid$

$= 0 + 0 + 0{,}30 = 0{,}30$

Keterangan: $\alpha \cdot \mid T \mid = 0{,}30$, sesuai dengan parameter penalti pada model.

**Tahap 4: Simulasi *Pruning* Menjadi *Leaf* Tunggal**

Jika kedua *leaf* digabung menjadi satu *leaf* baru, rata-rata baru dihitung sebagai:

$$\hat{y}_{\text{baru}} = \frac{833.627{,}4648 + 800.829{,}7397}{2} = 817.228{,}6023$$

Kuadrat selisih masing-masing nilai aktual terhadap rata-rata baru:

$$(833.627{,}46 - 817.228{,}6023)^2 = 268.336.059{,}92$$

$$(800.829{,}74 - 817.228{,}6023)^2 = 268.336.059{,}92$$

Sehingga total RSS untuk *leaf* tunggal:

$$\text{RSS total} = 268.336.059{,}92 + 268.336.059{,}92 = 536.672.119{,}84$$

Dengan penambahan penalti $\alpha \cdot 1 \approx 1 \cdot 0{,}1 = 0{,}1$, diperoleh Loss setelah prune ≈ 537.845.384,84, sesuai *output* model.

**Tahap 5: Evaluasi Keputusan Pruning**

*Loss* setelah *prune* (537.845.384,84) lebih besar dibandingkan loss sebelum prune (0,30)

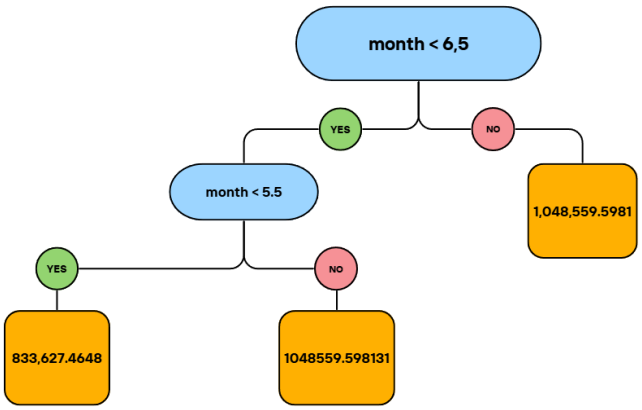
Dengan demikian, melakukan *pruning* tidak menguntungkan

**Contoh Inferensi**

Misalkan terdapat data baru dengan atribut:

Tabel 2.10 Dataset Untuk Inferensi

| *month* | *lag_1* | *price* |
|---|---|---|
| 8 | 1.048.560 | – |




Gambar 2.6 *Decision Tree* Akhir

Berdasarkan pohon keputusan yang telah dibangun:

1. Evaluasi *Root Node*:

   *Node root* melakukan pemisahan berdasarkan kriteria *month* ≤ 6,5. Karena nilai *month* pada data baru adalah 8, sampel ini masuk ke cabang kanan.

2. Evaluasi *Node* Kanan:

   Data diteruskan ke *node* kanan, yang merupakan *leaf* dengan nilai prediksi 1.048.559,5981

3. Prediksi Akhir:

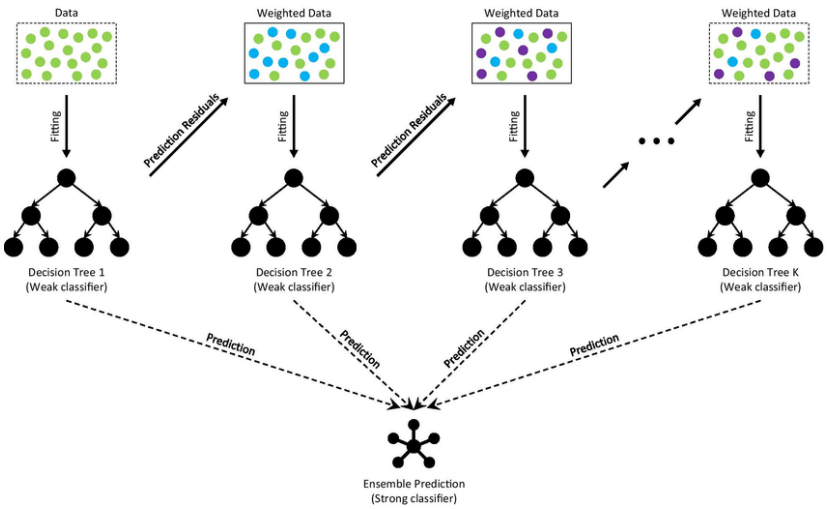
   Dengan demikian, nilai prediksi untuk data baru adalah 1.048.559,5981, sesuai dengan *leaf* tempat data tersebut berada.

### *2.2.4 Gradient Boosting*

Teknik pemodelan *gradient boosting* tampaknya beroperasi berdasarkan prinsip dasar *ensemble learning*, di mana sejumlah model prediksi berkapasitas rendah (*weak learners*) diintegrasikan secara sekuensial untuk membentuk sebuah fungsi prediktor yang solid. Dalam mayoritas kerangka kerja komputasi ini, struktur pohon keputusan (*decision trees*) kemungkinan besar dipilih sebagai pembelajar

dasar. Berlainan dengan metode *bagging* yang menumbuhkan setiap pohon secara independen, pendekatan *boosting* membangun model secara bertahap. Setiap pohon baru yang ditambahkan diduga kuat bertugas untuk mengoreksi galat sisa dari kombinasi fungsi pohon-pohon yang telah dibangun pada iterasi sebelumnya (Gonçalves dan Carvalho, 2025).

Gambar 2.7 Arsitektur *Gradient Boosting* (Deng dkk., 2021)




Secara konseptual, pengembangan model ini mendekati masalah estimasi fungsi dari sudut pandang optimasi numerik di dalam ruang fungsi (*function space*). Pendekatan tersebut didasarkan pada asumsi bahwa kombinasi aditif dari sekumpulan model berkinerja rendah dapat meminimalkan nilai harapan dari fungsi kerugian (*loss function*), yang dinotasikan sebagai $L\big(y, F(x)\big)$. Pada kasus regresi, seperti peramalan rata-rata harga kamar, fungsi kerugian kuadrat terkecil (*Mean Squared Error*) umumnya lazim digunakan. Apabila solusi minimisasi untuk fungsi kerugian spesifik dirasa sulit diselesaikan secara langsung, algoritma ini memanfaatkan strategi penurunan gradien (*steepest descent*) untuk memperbarui model (Gonçalves dan Carvalho, 2025).

Diketahui terdapat sebuah fungsi aproksimasi pada iterasi sebelumnya, $F_{m-1}(x)$. Fungsi prediktor yang baru, disimbolkan dengan $h(x; a_m)$, dapat diposisikan sebagai langkah *greedy* terbaik menuju estimasi fungsi sasaran $F^*(x)$, dengan batasan bahwa arah langkah tersebut berada dalam ruang parameter $h(x; a)$.

Arah langkah ini dihitung menggunakan gradien negatif dari fungsi kerugian terhadap prediksi saat ini. Secara matematis, bentuk gradien negatif tanpa batasan tersebut diekspresikan pada Persamaan 2.7:

$$-g_{im} = -\left[\frac{\partial L\big(y_i, F(x_i)\big)}{\partial F(x_i)}\right]_{F(x)=F_{m-1}(x)} \tag{2.8}$$

Vektor gradien ini hanya terdefinisi secara khusus pada titik-titik data observasi $x_i$, algoritma memerlukan proses generalisasi agar dapat diterapkan pada nilai $x$ lainnya. Salah satu kemungkinan penyelesaiannya adalah dengan memilih anggota dari kelas parameter $h(x; a)$ yang memiliki korelasi tertinggi atau paling sejajar dengan vektor gradien $-g_{im}$ di seluruh sebaran data. Optimasi ini dapat dieksekusi melalui minimisasi fungsi kuadrat terkecil terhadap "*pseudo-residual*" (nilai respons semu), sebagaimana ditunjukkan pada Persamaan 2.8:

$$a_m = \arg\min_{a} \sum_{i=1}^{N} \big(-g_{im} - \beta h(x_i; a)\big)^2 \tag{2.9}$$

Setelah parameter optimal $a_m$ didapatkan, nilai fungsi turunan ini digunakan untuk menggantikan gradien tak berbatas dalam strategi *steepest descent*. Tahapan berikutnya memerlukan pencarian garis proyektif (*line search*) untuk menemukan panjang langkah atau pembobot mutlak $\rho_m$ yang paling meminimalkan fungsi kerugian awal. Perhitungannya direpresentasikan pada Persamaan 2.9:

$$\rho_m = \arg\min_{\rho} \sum_{i=1}^{N} L\left(y_i, F_{m-1}(x_i) + \rho h(x_i; a_m)\right) \tag{2.10}$$

Pada akhirnya, model hampiran akan diperbarui secara aditif dengan menambahkan fungsi pembelajar baru yang telah dikalikan dengan bobot langkahnya:

$$F_m(x) = F_{m-1}(x) + \rho_m h(x; a_m) \tag{2.11}$$

Keseluruhan mekanisme matematis di atas dapat dirangkum ke dalam sebuah struktur algoritma komputasi yang berulang. Susunan operasional untuk prosedur *Gradient Boost* secara umum diuraikan melalui tahapan iteratif berikut (Gonçalves dan Carvalho, 2025):

Algoritma 1 (*Gradient Boost*)

1. Lakukan inisialisasi model awal menggunakan estimasi nilai konstan yang meminimalkan fungsi kerugian:

$$F_0(x) = \arg\min_{\rho} \sum_{i=1}^{N} L(y_i, \rho) \tag{2.12}$$

2. Untuk proses iterasi dari $m = 1$ hingga $M$, jalankan serangkaian komputasi pada langkah 3 sampai 6:
3. Hitung nilai *pseudo-residual* untuk setiap baris observasi

$$\tilde{y}_i = -\left[\frac{\partial L\big(y_i, F(x_i)\big)}{\partial F(x_i)}\right]_{F(x)=F_{m-1}(x)}, \quad i = 1, \dots, N \tag{2.13}$$

4. Latih model pembelajar dasar (contohnya pohon regresi) untuk memprediksi *residual* tersebut:

$$a_m = \arg\min_{a} \sum_{i=1}^{N} [\tilde{y}_i - h(x_i; a)]^2 \tag{2.14}$$

5. Lakukan pencarian nilai skalar optimal yang meminimalkan total kerugian pada prediksi yang diperbarui:

$$\rho_m = \arg\min_{\rho} \sum_{i=1}^{N} L\big(y_i, F_{m-1}(x_i) + \rho h(x_i; a_m)\big) \tag{2.15}$$

6. Perbarui model prediksi utama:

$$F_m(x) = F_{m-1}(x) + \rho_m h(x; a_m) \tag{2.16}$$

7. Akhiri perulangan.

Melalui penjabaran konsep dan turunan algoritma tersebut, terlihat bahwa *gradient boosting* memungkinkan pendekatan optimasi fungsi yang luwes. Alih-alih merumuskan optimasi yang rumit dalam satu waktu, teknik ini menerjemahkannya ke dalam persoalan minimisasi kesalahan kuadrat secara

bertahap, menjadikannya sangat relevan untuk menangkap pola deret waktu non-linear yang kompleks seperti dinamika rata-rata harga kamar hotel.

Dalam *Gradient Boosting*, *decision tree* biasanya digunakan sebagai *weak learner*. Jika kita menggunakan *decision tree* sebagai *weak learner* dan menggunakan *squared error loss*, maka persamaan *update* model dapat dituliskan sebagai berikut:

Algoritma 2 (*Gradient Boost* dengan *Decision Tree*)

1. Lakukan inisialisasi model awal menggunakan estimasi nilai konstan yang meminimalkan fungsi kerugian:

$$F_0(x) = \arg\min_{\rho} \sum_{i=1}^{N} L(y_i, \rho) \tag{2.17}$$

$$= \arg\min_{\rho} \sum_{i=1}^{N} \frac{1}{2}(y_i - \rho)^2 \tag{2.18}$$

$$= \arg\min_{\rho} \frac{1}{2} \sum_{i=1}^{N} (\rho - y_i)^2 \tag{2.19}$$

$$\Rightarrow \frac{d}{d\rho} \sum_{i=1}^{N} (\rho - y_i) = 0 \tag{2.20}$$

$$\Rightarrow \rho = \frac{1}{N} \sum_{i=1}^{N} y_i \tag{2.21}$$

$$\Rightarrow F_0(x) = \frac{1}{N} \sum_{i=1}^{N} y_i \tag{2.22}$$

2. Untuk proses iterasi dari $m = 1$ hingga $M$, jalankan serangkaian komputasi pada langkah 3 sampai 6:
3. Hitung nilai *pseudo-residual* untuk setiap baris observasi

$$\tilde{y}_i = -\left[\frac{\partial L\big(y_i, F(x_i)\big)}{\partial F(x_i)}\right]_{F(x)=F_{m-1}(x)}, \quad i = 1, \dots, N \tag{2.23}$$

$$L\big(y_i, F(x_i)\big) = \frac{1}{2}\big(y_i - F(x_i)\big)^2 \tag{2.24}$$

$$\frac{\partial L\big(y_i, F(x_i)\big)}{\partial F(x_i)} = -\big(y_i - F(x_i)\big) \tag{2.25}$$

4. Latih model pembelajar dasar (contohnya pohon regresi) untuk memprediksi *residual* tersebut:

$$a_m = \arg\min_a \sum_{i=1}^{N} [\widetilde{y_i} - h(x_i; a)]^2 \tag{2.26}$$

5. Lakukan pencarian nilai skalar optimal yang meminimalkan total kerugian pada prediksi yang diperbarui:

$$\rho_m = \arg\min_\rho \sum_{i=1}^{N} L\big(y_i, F_{m-1}(x_i) + \rho h(x_i; a_m)\big) \tag{2.27}$$

$$\begin{aligned} \rho_m &= \frac{\sum_{i=1}^{N} h(x_i; a_m)\big(y_i - F_{m-1}(x_i)\big)}{\sum_{i=1}^{N} h(x_i; a_m)^2}, \quad \text{dengan } L(y_i, F) \\ &= \frac{1}{2}(y_i - F)^2 \end{aligned} \tag{2.28}$$

6. Perbarui model prediksi utama dengan $v$ sebagai *learning rate* agar tidak *overfitting*:

$$F_m(x) = F_{m-1}(x) + v\rho_m h(x; a_m) \tag{2.29}$$

7. Akhiri perulangan.

**Contoh 2.4**

Misal dengan data sebagai berikut:

Tabel 2.11 Dataset Untuk *Gradient Boosting*

| *month* | *lag_1* | *price* |
|---|---|---|
| 5 | 406.650 | 833.627,464789 |
| 6 | 833.627,464789 | 800.829,739726 |

**Langkah 1: Lakukan inisialisasi**

$$F_0(x) = \frac{833.627{,}464789 + 800.829{,}739726}{2} = \frac{1.634.457{,}2045147596}{2}$$
$$= 817.228{,}6023$$

**Langkah 2: Ulangi Langkah 3-6 Hingga M kali**

**Langkah 3: Menghitung *pseudo-residual***

$$\tilde{y} = \begin{bmatrix} 833.627,46 \\ 800.829,74 \end{bmatrix} - \begin{bmatrix} 817.228,60 \\ 817.228,60 \end{bmatrix} = \begin{bmatrix} 16.398,86 \\ -16.398,86 \end{bmatrix}$$

**Langkah 4: Melatih *weak learner* ke *psuedo-residual***

**1. *Node* pada kedalaman 1:**

Tabel 2.12 Dataset Latih *Weak Learner*

| *month* | *lag_1* | *psuedo-residual* |
|---|---|---|
| 5 | 406.650 | 16.398,862531 |
| 6 | 833.627.464789 | -16.398,862531 |

Untuk j=1:

$$s = \frac{5+6}{2} = 6$$

$$R_1(1,6) = \{x_1\}$$

$$R_2(1,6) = \{x_2\}$$

$$RSS(1,5.5) = (16.398,86 - 16.398,8625)^2 + (-16.398,86 - -16.398,8625)^2$$
$$= 0,00$$

$$s = \frac{406.650 + 833.627}{2} = 620.139$$

$$R_1(2,620.139) = \{x_1\}$$

$$R_2(2,620.139) = \{x_2\}$$

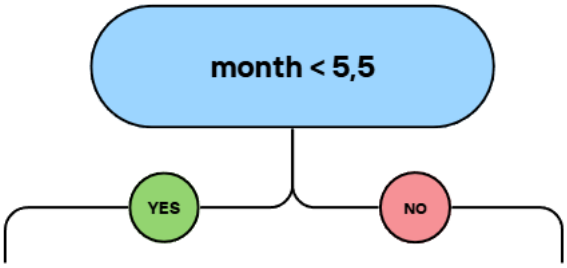
$$RSS(2,620.138.73)$$
$$= (16.398,86 - 16.398,8625)^2 + (-16.398,86$$
$$- -16.398,8625)^2 = 0,00$$

Jumlah kuadrat selisih (RSS) untuk *node* ini dihitung sebagai:

$$(16,398.86 - 16,398.8625)^2 + (-16,398.86 - -16,398.8625)^2 = 0$$

Berdasarkan perhitungan ini, *split* yang dipilih menggunakan fitur *month* dengan *threshold* 5,5, menghasilkan RSS sebesar 0.

Gambar 2.8 *Weak Learner Root*




**2. *Node* pada kedalaman 2:**

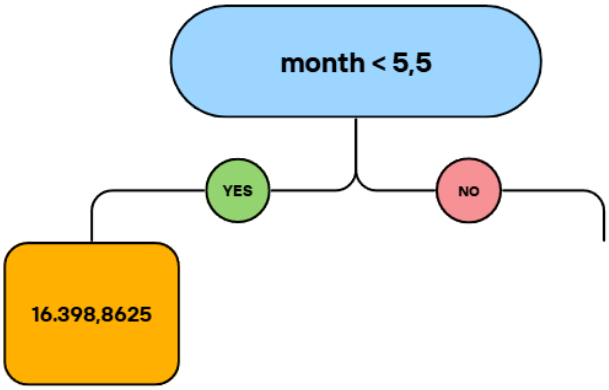
Selanjutnya, pohon dilanjutkan pada *node* kiri dengan data sebagai berikut:

Tabel 2.13 Dataset *Weak Learner Level* 2 *Node* Kiri

| *month* | *lag_1* | *psuedo-residual* |
|---|---|---|
| 5 | 406.650 | 16.398,862531 |

Karena *node* ini hanya memiliki satu sampel, *node* ini dijadikan *leaf* dengan nilai prediksi = 16.398,8625.

Gambar 2.9 *Weak Learner Level* 1 *Node* Kiri



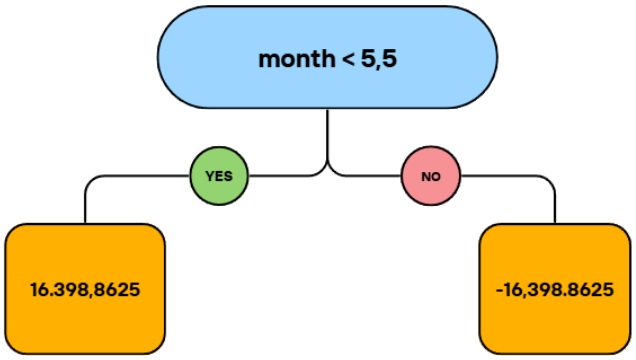

Selanjutnya, pohon dilanjutkan pada *node* kanan dengan data sebagai berikut:

Tabel 2.14 Dataset *Weak Learner Level* 2 *Node* Kanan

| *month* | *lag_1* | *psuedo-residual* |
|---|---|---|
| 6 | 833.627,464789 | -16.398,862531 |

Karena *node* ini hanya memiliki satu sampel, *node* ini dijadikan *leaf* dengan nilai prediksi = -16.398,8625.

Gambar 2.10 *Weak Learner Level* 1 *Node* Kanan




*Pruning* tidak diperlukan karena kedalaman pohon terbatas.

**Langkah 4: Mencari $\rho_1$**

$$\rho_1 = \frac{\sum_{i=1}^{N} h(x_i; a_1)\big(y_i - F_0(x_i)\big)}{\sum_{i=1}^{N} h(x_i; a_1)^2},$$

Berdasarkan $p_1$ kita perlu $\sum_{i=1}^{N} h(x_i; a_1)$ yaitu *output* setiap dataset menggunakan *weak learner* yang baru dibuat sebagai berikut:

Tabel 2.15 Dataset Untuk Prediksi *Weak Learner*

| *month* | *lag_1* |
|---|---|
| 5 | 406.650 |
| 6 | 833.627,464789 |

1. Data 1: *month* = 5, *lag_1*= 406.650

   Karena nilai bulan kurang dari 5,5, keluaran dari basis fungsi pohon untuk data pertama adalah:

$$h(x_1; a_1) = +16.398{,}862531$$

2. Data 2: *month* = 6, *lag_1*= 833.627,464789

   Karena nilai bulan lebih dari 5,5, keluaran dari basis fungsi pohon untuk data kedua adalah:

$$h(x_2; a_1) = -16.398{,}862531$$

Maka dengan $\left(y_i - F_0(x_i)\right)$ sebagai *psuedo-residual*,

$$\begin{aligned}
\rho_m &= \frac{\sum_{i=1}^{2} h(x_i; a_1)\left(y_i - F_0(x_i)\right)}{\sum_{i=1}^{2} h(x_i; a_1)^2} \\
&= \frac{h(x_1; a_1)\left(y_1 - F_0(x_1)\right) + h(x_2; a_1)\left(y_2 - F_0(x_2)\right)}{h(x_1; a_1)^2 + h(x_2; a_1)^2} \\
&= \frac{16.398{,}86 \times 16.398{,}86 + (-16.398{,}86) \times (-16.398{,}86)}{16.398{,}86^2 + (-16.398{,}86)^2} \\
&= \frac{268.752.905{,}45 + 268.752.905{,}45}{268.752.905{,}45 + 268.752.905{,}45} \\
&= \frac{537.505.810{,}9}{537.505.810{,}9} \\
&= 1
\end{aligned}$$

**Langkah 6: Perbarui model prediksi utama**

Dengan asumsi v = 0,1, diperoleh,

$$F_1(x) = F_0(x) + v\rho_1 h(x; a_1)$$

$$F_1(x) = 817.228{,}6023 + 0{,}1 \times 1 \times h(x; a_1)$$

Di sini, $h(x; a_1)$ merepresentasikan *weak learner* berupa *decision tree* yang baru saja dibuat. Dalam contoh ini, hanya dibuat satu pohon, atau dengan kata lain $M = 1$. Namun, pada praktiknya, biasanya $M > 1$, sehingga langkah 3–6 pada algoritma *gradient boosting* dijalankan berulang kali. Pada langkah ke-6 setiap iterasi, model prediksi $F_m(x)$ diperbarui dengan menambahkan $v\rho_m h(x; a_m)$ ke model prediksi sebelumnya $F_{m-1}(x)$, dan proses ini terus diulang hingga seluruh *M weak learner* selesai ditambahkan.

**Langkah 7: Akhiri perulangan**

Setelah seluruh iterasi sebanyak $M$ dijalankan, model telah terbentuk dan

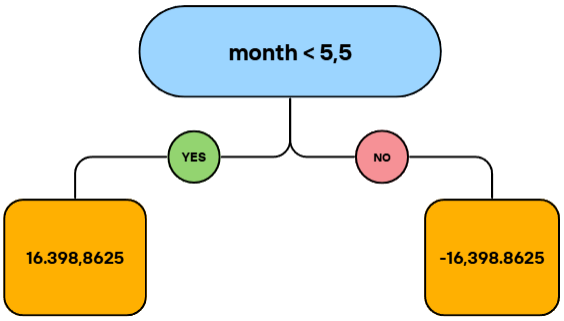
siap digunakan untuk proses inferensi. Sebagai contoh, pertimbangkan data pengujian yang disajikan pada Tabel 2.14:

Tabel 2.16 Dataset Untuk Testing

| *month* | *lag_1* |
|---|---|
| 7 | 800.829,739726 |

Dengan data seperti di atas, proses prediksi dapat dilakukan sebagai berikut,

Gambar 2.11 *Weak Learner* Akhir




Berdasarkan *weak learner* terakhir untuk $m = 1$, karena nilai *month* = 7 lebih besar dari 5,5, diperoleh:

$$h(x; a_1) = -16.398{,}862531$$

Dengan demikian, persamaan model prediksi setelah iterasi pertama menjadi:

$$\begin{aligned} F_1(x) &= F_0(x) + \nu\, \rho_1\, h(x; a_1) \\ &= 817.228{,}6023 + 0{,}1 \times 1 \times (-16.398{,}862531) \\ &= 817.228{,}6023 - 1.639{,}8862531 \\ &= 815.588{,}7167 \end{aligned}$$

Sehingga hasil prediksi pertama $F_1(x)$ adalah 815.588,7167.

### 2.2.5 *Light Gradient Boosting Machine* (LightGBM)

*Light Gradient Boosting Machine* (LightGBM) pada dasarnya merupakan kerangka kerja pembelajaran mesin yang mengadopsi prinsip *gradient boosting* dengan memanfaatkan pohon keputusan sebagai pembelajar dasar (Gonçalves dan Carvalho, 2025). Kehadiran algoritma ini diduga kuat berawal dari kebutuhan

praktis untuk mengatasi kendala komputasi pada metode *boosting* konvensional, terutama ketika dihadapkan pada dimensi data yang masif. Berbeda dengan pendekatan tradisional, algoritma ini menawarkan efisiensi memori dan waktu pelatihan yang jauh lebih ringkas tanpa harus mengorbankan akurasi secara drastis (Indah dkk., 2025).

Salah satu inovasi utama yang mendasari efisiensi operasional tersebut tampaknya terletak pada pemanfaatan algoritma berbasis histogram. Pada model konvensional, penentuan titik belah (*split point*) mengharuskan sistem untuk memindai seluruh nilai fitur kontinu secara berurutan. Sebaliknya, pendekatan histogram dalam algoritma ini mengelompokkan nilai fitur tersebut ke dalam sejumlah *bin* diskret (Indah dkk., 2025). Proses pembentukan histogram ini diawali dengan menginisialisasi himpunan simpul pohon dan indeks data pelatihan. Untuk setiap kedalaman pohon, algoritma memindai himpunan data yang aktif dan membangun histogram dengan mengakumulasi nilai gradien target serta jumlah kemunculan observasi pada setiap *bin*. Langkah ini pada praktiknya mereduksi kompleksitas komputasi secara signifikan, mengingat evaluasi titik belah hanya perlu dilakukan pada sekumpulan *bin* tersebut, bukan pada keseluruhan observasi secara individual (Gonçalves dan Carvalho, 2025).

Secara matematis, evaluasi kualitas suatu titik belah pada algoritma LightGBM didasarkan pada besarnya penurunan fungsi kerugian (*loss reduction*) yang dihasilkan oleh pemisahan tersebut. Nilai penurunan kerugian ini umumnya dinyatakan dalam bentuk gain, yang dihitung berdasarkan akumulasi gradien orde pertama dan kedua dari fungsi kerugian pada setiap simpul pohon. Secara umum, nilai gain dapat dirumuskan sebagai berikut:

$$\text{Gain} = \frac{1}{2}\left[\frac{\left(\sum_{i\in I_L} g_i\right)^2}{\sum_{i\in I_L} h_i + \lambda} + \frac{\left(\sum_{i\in I_R} g_i\right)^2}{\sum_{i\in I_R} h_i + \lambda} - \frac{\left(\sum_{i\in I} g_i\right)^2}{\sum_{i\in I} h_i + \lambda}\right] - \gamma \tag{2.30}$$

Keterangan:

$g_i$: gradien orde pertama, yaitu turunan pertama dari fungsi kerugian (*loss function*) terhadap nilai prediksi pada observasi ke-$i$.

$h_i$: gradien orde kedua (*Hessian*), yaitu turunan kedua dari fungsi kerugian yang merepresentasikan kelengkungan fungsi pada observasi ke-$i$.
$I$: himpunan seluruh observasi yang berada pada simpul (leaf) sebelum proses pemisahan dilakukan.
$I_L$: himpunan observasi yang akan dialokasikan ke simpul kiri setelah proses pemisahan (*split*).
$I_R$: himpunan observasi yang akan dialokasikan ke simpul kanan setelah proses pemisahan.
$\lambda$: parameter regularisasi L2 yang digunakan untuk mengendalikan kompleksitas model dengan memberikan penalti terhadap bobot daun yang terlalu besar sehingga dapat mengurangi risiko *overfitting*.
$\gamma$: ambang batas minimum nilai gain yang diperlukan agar suatu pemisahan simpul dapat dilakukan. Apabila nilai gain yang diperoleh lebih kecil dari $\gamma$, maka proses pemisahan tidak akan dilakukan.

Selain optimasi pada proses pencarian titik belah, arsitektur model ini juga dikenal dengan strategi pertumbuhan pohonnya yang berorientasi pada daun (*leaf-wise*). Mayoritas algoritma pohon keputusan tradisional cenderung menumbuhkan cabangnya secara simetris atau melebar (*level-wise*). Akan tetapi, strategi *leaf-wise* justru menyeleksi daun yang menjanjikan penurunan kerugian (*max delta loss*) paling besar untuk dimekarkan lebih lanjut. Mekanisme pemekaran tak simetris ini berpotensi menekan tingkat kesalahan prediktif dengan lebih tajam, meskipun pada ukuran himpunan data yang terlampau kecil, metode ini cukup rentan memicu *overfitting* apabila parameter kedalaman pohon tidak dikendalikan dengan ketat (Indah dkk., 2025).

Kemampuan arsitektur ini dalam memproses data berdimensi tinggi semakin diperkuat oleh kehadiran dua teknik optimasi bawaan, yakni *Gradient-based One-Side Sampling* (GOSS) dan *Exclusive Feature Bundling* (EFB). Konsep GOSS bekerja dengan menyeleksi sampel data pelatihannya berdasarkan nilai residual. Asumsi dasarnya adalah bahwa himpunan data dengan nilai gradien kecil kemungkinan besar telah direpresentasikan dengan baik oleh model. Oleh karena

itu, mekanisme GOSS mempertahankan seluruh instansi data yang belum dipelajari dengan baik (bergradien tinggi), sembari hanya mengambil sampel acak pada kelompok data bergradien rendah (Gonçalves dan Carvalho, 2025). Penyesuaian bobot konstan kemudian diterapkan pada sampel acak tersebut agar distribusi data secara keseluruhan tidak menyimpang dari populasi aslinya.

Di sisi lain, teknik EFB berupaya merampingkan dimensi ruang fitur. Berbagai kumpulan data riil di industri, seperti hasil penyandian identitas properti hotel ataupun tipe kamar, sering kali membentuk matriks yang sangat jarang (*sparse*). Fitur-fitur eksklusif yang jarang bernilai bukan nol secara bersamaan ini pada akhirnya digabungkan oleh algoritma EFB menjadi sebuah fitur tunggal yang padat. Penggabungan semacam ini terbukti mempercepat laju komputasi pada ruang dimensi data yang luas (Gonçalves dan Carvalho, 2025).

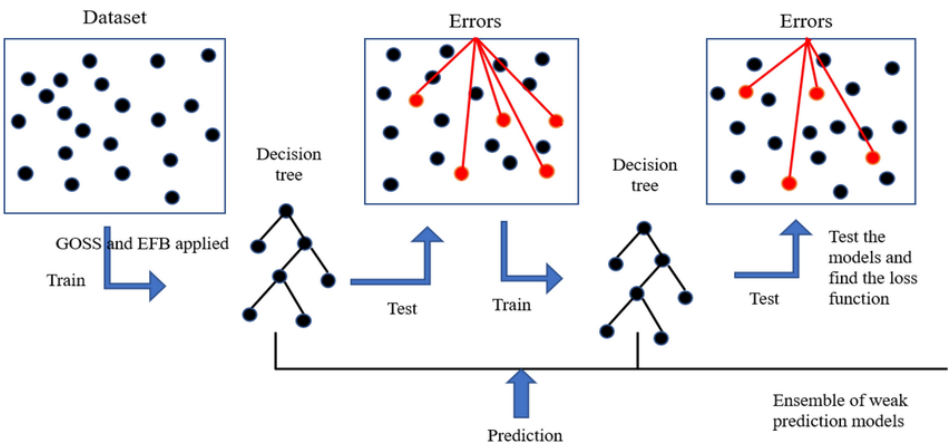
Pada tahap akhir pelatihannya, model dibangun secara bertahap melalui proses iteratif dengan menambahkan kontribusi dari setiap pembelajar dasar yang dihasilkan. Pada setiap iterasi ke-$m$, model sebelumnya diperbarui dengan menambahkan fungsi prediksi dari pohon keputusan baru yang telah dikalibrasi melalui parameter pembobot. Kontribusi pembaruan tersebut juga dikendalikan oleh parameter *learning rate* untuk mengatur besarnya langkah pembelajaran sehingga proses pembelajaran menjadi lebih stabil dan mengurangi risiko *overfitting*. Dengan demikian, pembaruan model pada iterasi ke-$m$ dapat dinyatakan dalam bentuk persamaan rekursif sebagai berikut.

$$F_m(x) = F_{m-1}(x) + \nu h(x; a_m) \tag{2.31}$$

Dalam rumusan persamaan tersebut, $F_m(x)$merepresentasikan nilai prediksi model pada iterasi ke-$m$ setelah dilakukan proses pembaruan. Sementara itu, $F_{m-1}(x)$ menunjukkan nilai prediksi model pada iterasi sebelumnya. Fungsi $h(x; a_m)$ menyatakan fungsi prediksi dari pohon keputusan sebagai pembelajar dasar pada iterasi ke-$m$ dengan parameter $a_m$. Parameter $\nu$ merupakan *learning rate* yang mengatur besarnya langkah pembaruan model pada setiap iterasi. Dengan mekanisme ini, setiap pembelajar dasar memberikan kontribusi bertahap dalam meningkatkan akurasi prediksi model secara keseluruhan (Indah dkk., 2025).

Melalui kombinasi strategi penyederhanaan titik belah berbasis histogram, pertumbuhan pohon yang berfokus pada kerugian daun, serta reduksi dimensi fitur, algoritma LightGBM mampu menghadirkan stabilitas komputasi yang tinggi. Karakteristik teknis tersebut menjadikannya sangat relevan dan rasional untuk diimplementasikan dalam menangani kerumitan pemodelan deret waktu yang melibatkan banyak variabel kategorikal prediktif.

Gambar 2.12 Arsitektur LightGBM (Bhupatiraju, 2025)




Berikut disajikan ringkasan algoritma model *Light Gradient Boosting Machine* (LightGBM) yang menggunakan fungsi kerugian *squared error* untuk mengukur selisih antara nilai aktual dan nilai prediksi model. Fungsi kerugian tersebut dinyatakan sebagai berikut.

$$L\big(y_i, F(x_i)\big) = \frac{1}{2}\big(y_i - F(x_i)\big)^2 \tag{2.32}$$

Selanjutnya, tahapan algoritma LightGBM dapat diringkas sebagai berikut:

Algoritma 3 (LightGBM dengan *Squared Error Loss*)

1. Lakukan inisialisasi model awal menggunakan estimasi nilai konstan yang meminimalkan fungsi kerugian:

$$F_0(x) = \arg\min_{\rho} \sum_{i=1}^{N} L(y_i, \rho) \tag{2.33}$$

$$\Rightarrow \frac{d}{d\rho}\left(\frac{1}{2}\sum_{i=1}^{N}(\rho - y_i)^2\right) = 0 \tag{2.34}$$

$$\Rightarrow \sum_{i=1}^{N}(\rho - y_i) = 0 \tag{2.35}$$

$$\Rightarrow N\rho - \sum_{i=1}^{N} y_i = 0 \tag{2.36}$$

$$\Rightarrow \rho = \frac{1}{N}\sum_{i=1}^{N} y_i \tag{2.37}$$

$$\Rightarrow F_0(x) = \frac{1}{N}\sum_{i=1}^{N} y_i \tag{2.38}$$

2. Lakukan transformasi fitur menggunakan *histogram binning* dan *Exclusive Feature Bundling* (EFB). Proses EFB dilakukan melalui Algoritma 4 (*Greedy Bundling*) dan Algoritma 5 (*Merge Exclusive Features*).
3. Melakukan Transformasi Fitur Menggunakan Histogram Binning:

   *Histogram binning* merupakan teknik diskretisasi fitur kontinu dengan cara membagi rentang nilai setiap fitur ke dalam sejumlah interval (bin) diskret.

   Secara matematis, misalkan dataset fitur dinyatakan sebagai matriks:

$$X \in \mathbb{R}^{N\times m} \tag{2.39}$$

   Dengan $N$ menyatakan jumlah observasi dan $m$ menyatakan jumlah fitur. Untuk setiap fitur ke-$j$, ditentukan nilai minimum dan maksimum sebagai:

$$x_{min}^{(j)} = \min_{i=1,\dots,N} x_{ij} \tag{2.40}$$

$$x_{max}^{(j)} = \max_{i=1,\dots,N} x_{ij} \tag{2.41}$$

   Jika jumlah bin yang digunakan adalah $B$, maka lebar setiap bin dihitung sebagai:

$$w_j = \frac{x_{max}^{(j)} - x_{min}^{(j)}}{B} \tag{2.42}$$

   Batas interval setiap bin (bin *edges*) kemudian diperoleh dengan:

$$e_k^{(j)} = x_{min}^{(j)} + k \cdot w_j, k = 0,1, \dots, B \tag{2.43}$$

Sehingga terbentuk himpunan batas bin,

$$E^{(j)} = \{e_0^{(j)}, e_1^{(j)}, \dots, e_B^{(j)}\} \tag{2.44}$$

Setiap nilai fitur kemudian dipetakan ke indeks bin menggunakan fungsi pemetaan:

$$b_{ij} = \text{digitize}\left(x_{ij}, e_1^{(j)}, e_2^{(j)}, \dots, e_{B-1}^{(j)}\right) \tag{2.45}$$

Hasil transformasi ini menghasilkan matriks fitur baru:

$$X_{\text{bin}} \in \mathbb{Z}^{N \times m} \tag{2.46}$$

Matriks hasil transformasi ini memiliki dimensi yang sama dengan dataset fitur awal, namun setiap elemennya merupakan indeks bin dari nilai fitur tersebut. Representasi diskrit ini kemudian digunakan dalam proses komputasi akumulasi *gradient* dan *hessian* pada algoritma pembelajaran berbasis histogram.

4. Untuk proses iterasi dari $m = 1$ hingga $M$, jalankan komputasi pada langkah 5 sampai 7:
5. Hitung nilai turunan pertama (*gradient*) dan turunan kedua (*hessian*) dari fungsi kerugian terhadap prediksi model:

$$g_i = \frac{\partial L\big(y_i, F(x_i)\big)}{\partial F(x_i)}, \quad i = 1, \dots, N \tag{2.47}$$

$$h_i = \frac{\partial^2 L\big(y_i, F(x_i)\big)}{\partial F(x_i)^2}, \quad i = 1, \dots, N \tag{2.48}$$

Di mana $F(x_i)$ merupakan nilai prediksi pada iterasi sebelumnya, yaitu $F_{m-1}(x)$.

Dengan *loss* $L\big(y_i, F(x_i)\big) = \frac{1}{2}\big(y_i - F(x_i)\big)^2$ maka,

$$g_i = \frac{\partial}{\partial F(x_i)}\left[\frac{1}{2}\big(y_i - F(x_i)\big)^2\right] \tag{2.49}$$

$$= -\big(y_i - F(x_i)\big) \tag{2.50}$$

$$= F(x_i) - y_i \tag{2.51}$$

$$h_i = \frac{\partial^2}{\partial F(x_i)^2}\left[\frac{1}{2}\big(y_i - F(x_i)\big)^2\right] = 1 \tag{2.52}$$

6. Bangun pohon keputusan dengan meminimalkan fungsi objektif yang telah diaproksimasi menggunakan deret Taylor orde dua:

$$L^{(t)} = \sum_{j=1}^{T} \left[ \sum_{i \in I_j} g_i w_j + \frac{1}{2} \left( \sum_{i \in I_j} h_i + \lambda \right) w_j^2 + \alpha |w_j| \right] + \gamma T \tag{2.53}$$

di mana:

$T$: jumlah daun pada pohon

$I_j$: himpunan observasi pada daun ke-$j$

$w_j$: bobot prediksi pada daun ke-$j$

$\lambda$: parameter regularisasi $L_2$

$\alpha$: parameter regularisasi $L_1$

$\gamma$: penalti kompleksitas pohon

Melalui bentuk fungsi objektif tersebut, bobot optimal pada setiap daun dapat diperoleh secara langsung sebagai:

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} \tag{2.54}$$

Persamaan tersebut berlaku ketika *node* berperan sebagai daun. Adapun untuk menentukan titik pemisahan yang optimal pada setiap *node*, digunakan perhitungan nilai *gain*:

$$\text{Gain} = \frac{1}{2} \left[ \frac{\left(\sum_{i \in I_L} g_i\right)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{\left(\sum_{i \in I_R} g_i\right)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{\left(\sum_{i \in I} g_i\right)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma \tag{2.55}$$

di mana:

$I_L$: himpunan observasi pada cabang kiri

$I_R$: himpunan observasi pada cabang kanan

$I$: himpunan observasi sebelum pemisahan

Nilai *gain* terbesar menunjukkan titik pemisahan yang paling optimal dalam meningkatkan kualitas model.

Data yang digunakan pada tahap ini terdiri atas observasi dengan rasio $a$ dari gradien terbesar serta rasio $b$ dari gradien lainnya yang dipilih sebagai sampel. Untuk menjaga agar distribusi data tetap tidak bias, sampel

dengan rasio $b$ tersebut diberikan faktor penyesuaian sebesar $\frac{1-a}{b}$.

7. Perbarui model prediksi utama dengan *learning rate* $\nu$:

$$F_m(x) = F_{m-1}(x) + \nu h(x; a_m) \tag{2.56}$$

   Komponen $h(x; a_m)$ merupakan pohon keputusan yang dihasilkan pada iterasi ke-$m$.

8. Akhiri perulangan ketika jumlah iterasi $M$ telah tercapai.

Algoritma 4 (*Greedy Bundling*)

Gambar 2.13 *Algoritma Greedy Bundling*

```
Input  : F = fitur, K = jumlah konflik maksimum
Output : bundles

Bangun graf G
searchOrder ← G.sortByDegree()

bundles ← {}
bundlesConflict ← {}

for i dalam searchOrder do
    needNew ← True
    for j = 1 sampai len(bundles) do
        cnt ← ConflictCnt(bundles[j], F[i])
        if cnt + bundlesConflict[i] ≤ K then
            bundles[j].add(F[i])
            needNew ← False
            break
    if needNew then
        Tambahkan F[i] sebagai bundle baru
```

Algoritma 5 (*Merge Exclusive Features*)

Gambar 2.14 Algoritma *Merge Exclusive Features*

```
Input  : numData = jumlah data
Input  : F = satu bundle fitur yang saling eksklusif
Output : newBin, binRanges

binRanges ← {0}
totalBin ← 0

for f dalam F do
    totalBin ← totalBin + f.numBin
    binRanges.append(totalBin)

newBin ← Bin baru dengan ukuran numData

for i = 1 sampai numData do
    newBin[i] ← 0
    for j = 1 sampai len(F) do
        if F[j].bin[i] ≠ 0 then
            newBin[i] ← F[j].bin[i] + binRanges[j]
```

**Contoh 2.5**

Sebagai ilustrasi, misalkan data diberikan sebagai berikut:

Tabel 2.17 Dataset Untuk LightGBM

| *month* | *lag_1* | *season_Low* | *season_Mid* | *season_High* | *season_Peak* | *price* |
|---|---|---|---|---|---|---|
| 5 | 406.650 | 0 | 1 | 0 | 0 | 833.627,464789 |
| 6 | 833.627,464789 | 0 | 1 | 0 | 0 | 800.829,739726 |

**Langkah 1: Lakukan inisialisasi model awal**

$$F_0(x) = \frac{1}{N}\sum_{i=1}^{N} y_i$$

$$= \frac{833.627,464789 + 800.829,739726}{2}$$

$$= \frac{1.634.457,2045147596}{2}$$

$$= 817.228,6022573798$$

**Langkah 2: Lakukan transformasi fitur dengan EFB**

***Greedy Bundling dengan K=1:***

Karena EFB bekerja paling efektif pada fitur yang bersifat *sparse*, kita hanya menggunakan kolom kategorikal yang telah diubah melalui *one-hot encoding* dan memiliki sifat *sparse*, seperti berikut:

Tabel 2.18 *Dataset Sparse*

| *season_Low* | *season_Mid* | *season_High* | *season_Peak* |
|---|---|---|---|
| 0 | 1 | 0 | 0 |
| 0 | 1 | 0 | 0 |

Langkah pertama adalah menghitung konflik, yaitu banyaknya observasi di mana dua fitur memiliki nilai non-nol secara bersamaan.

$$C = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Matriks konflik $C$ menunjukkan jumlah observasi di mana dua fitur aktif bersamaan, misalnya jika $C_{12} = 2$ berarti ada 2 observasi di mana *season_Low* dan *season_Mid* aktif bersamaan, sedangkan elemen lain bernilai 0 menunjukkan tidak ada konflik antara pasangan fitur tersebut, sehingga fitur-fitur lain dapat dibundle tanpa masalah dalam proses EFB.

Selanjutnya, dihitung *degree* yaitu jumlah konflik per fitur:

$$degree = [0,0,0,0]$$

Karena semua *degree* sama, pengurutan fitur tidak memengaruhi search order, sehingga:

$$searchOrder = [0,0,0,0]$$

Kemudian dibuat *bundle* dan bundleConflict awal:

$$bundle = [\quad]$$

$$bundleConflict = [\quad]$$

Lalu kita akan memproses setiap searchOrder

1. Memproses fitur 1

Karena belum ada *bundle*, fitur 0 ditambahkan ke bundle pertama:

$$bundle = [[1]]$$

$$bundleConflict = [0]$$

2. Memproses fitur 2

Untuk *bundle* 1:

Karena $C_{2,1} = 0$ maka,

$$cnt = 0$$

Selanjutnya, karena *cnt* + *bundleConflict*$[1] = 0 + 0 = 0 \leq K = 1$, fitur 2 ditambahkan ke bundle pertama:

$$bundle = [[1,2]]$$

Selanjutnya, nilai *cnt* dijumlahkan ke *bundleConflict*[1]

$$bundleConflict = [0 + 0] = [0]$$

3. Memproses fitur 3

Untuk *bundle* 1:

Karena $C_{3,1} = 0$ dan $C_{3,2} = 0$ maka,

$$cnt = 0 + 0 = 0$$

Selanjutnya, karena *cnt* + *bundleConflict*$[1] = 0 + 0 = 0 \leq K = 1$, fitur 3 ditambahkan ke *bundle* pertama:

$$bundle = [[1,2,3]]$$

Selanjutnya, nilai *cnt* dijumlahkan ke *bundleConflict*[1]

$$bundleConflict = [0 + 0] = [0]$$

4. Memproses fitur 4

Untuk *bundle* 1:

Karena $C_{3,1} = 0, C_{3,2} = 0, dan\ C_{3,3} = 0$ maka,

$$cnt = 0 + 0 + 0 = 0$$

Selanjutnya, karena *cnt* + *bundleConflict*$[1] = 0 + 0 = 0 \leq K = 1$, fitur 4 ditambahkan ke *bundle* pertama:

$$bundle = [[1,2,3,4]]$$

Selanjutnya, nilai *cnt* dijumlahkan ke *bundleConflict*[1]

$$bundleConflict = [0 + 0] = [0]$$

Karena fitur-fitur bersifat sangat *sparse*, seluruh fitur dapat digabung dalam satu *bundle*. Pada kondisi fitur yang kurang *sparse*, algoritma akan mengecek setiap *bundle* yang ada untuk fitur tertentu dan hanya menambahkannya jika *cnt* +

$bundleConflict$[bundle] $\leq K$, jika tidak fitur tersebut akan ditempatkan dalam bundle baru.

***Merge Exclusive Feature:***

**Untuk bundle 1:**

1. Mencari *binRange*

Untuk *Bundle* 1, pertama-tama dilakukan inisialisasi *binRange* dengan nilai awal [0]. Selanjutnya, dilakukan penambahan total bin per fitur dalam *bundle* secara kumulatif. Total bin setiap fitur dihitung berdasarkan jumlah nilai unik dalam kolom, di mana fitur biner dengan nilai 0 dan 1 hanya dihitung sebagai satu bin. Proses kumulatif ini dilakukan sebagai berikut:

*binRange* pertama [0] kemudian ditambahkan dengan total bin fitur pertama (0 + 1 = 1), menghasilkan *binRange* [0, 1]. Penambahan berikutnya mempertimbangkan total bin fitur kedua ditambah nilai terakhir *binRange* (1 + 1 = 2), sehingga *binRange* menjadi [0, 1, 2]. Proses yang sama diterapkan untuk fitur ketiga dan keempat, menghasilkan *binRange* akhir [0, 1, 2, 3, 4].

Dengan cara ini, *binRange* menunjukkan *offset* kumulatif untuk setiap fitur dalam *bundle* sehingga setiap nilai fitur dapat dipetakan ke posisi yang unik pada representasi fitur gabungan.

2. Iterasi tiap data

Setiap baris data dalam dataset diproses dengan melakukan iterasi terhadap fitur-fitur yang terdapat dalam sebuah *bundle*. Untuk setiap baris, dicari fitur pertama yang memiliki nilai bukan nol. Nilai fitur ini kemudian digunakan untuk menentukan nilai akhir pada dataset baru, di mana nilai akhir dihitung berdasarkan *binRange* yang sesuai dengan posisi fitur dalam *bundle*. Apabila semua fitur dalam *bundle* memiliki nilai nol, maka nilai akhir untuk baris tersebut tetap diatur menjadi nol.

Untuk mempermudah pemahaman, dataset *Bundle* 1 disajikan secara visual sebagai ilustrasi proses transformasi ini:

Tabel 2.19 Dataset Bundle 1

| Fitur 1 | Fitur 2 | Fitur 3 | Fitur 4 |
|---|---|---|---|
| 0 | 1 | 0 | 0 |
| 0 | 1 | 0 | 0 |

a. Untuk data 1:

Karena fitur kedua memiliki nilai bukan nol dan *binRange*[2] = 1, maka nilai akhir dihitung sebagai *val* = fitur + *binRange*[2] = 1 + 1 = 2, sehingga menghasilkan dataset baru sebagai berikut:

Tabel 2.20 Dataset Baru Untuk Data 1

| *bundle_1* |
|---|
| 2 |
| 0 |

b. Untuk data 2:

Karena fitur kedua memiliki nilai bukan nol dan *binRange*[2] = 1, maka nilai akhir dihitung sebagai *val* = fitur + *binRange*[2] = 1 + 1 = 2, sehingga menghasilkan dataset baru sebagai berikut:

Tabel 2.21 Dataset Baru Untuk Data 2

| *bundle_1* |
|---|
| 2 |
| 2 |

Dengan demikian, dataset yang menggabungkan fitur hasil *bundle* dengan fitur numerik awal dapat disajikan sebagai berikut:

Tabel 2.22 Dataset Gabungan

| *month* | *lag_1* | *bundle_1* | *price* |
|---|---|---|---|
| 5 | 406.650 | 2 | 833.627,464789 |
| 6 | 833.627 | 2 | 800.829,739726 |

**Langkah 3: Lakukan transformasi fitur *histogram binning***

Dataset memiliki 2 observasi dan 3 fitur**.**

$$X = \begin{bmatrix} 5 & 406.650 & 2 \\ 6 & 833.627.46478873 & 2 \end{bmatrix}$$

Misal jumlah bin yang digunakan adalah $B = 2$, maka setiap fitur akan memiliki 3 bin *edges***.**

1. Fitur 1:

Nilai minimum, maksimum, dan lebar bin pada fitur pertama dihitung sebagai berikut:

$$x^{(1)} = [5,6]$$

$$x_{min}^{(1)} = 5$$

$$x_{max}^{(1)} = 6$$

Lebar bin diperoleh dengan:

$$w_1 = \frac{6-5}{2} = 0,5$$

Selanjutnya ditentukan batas setiap bin sebagai berikut:

$$e_0^{(1)} = 5$$

$$e_1^{(1)} = 5 + 0,5 = 5,5$$

$$e_2^{(1)} = 6$$

Dengan demikian diperoleh batas bin (*edge*) dan interval sebagai berikut:

$$edge = [5; 5,5; 6]$$

$$Bin\ 0: [5; 5,5)$$

$$Bin\ 1: \ [5,5;\ 6]$$

Tahap berikutnya adalah memetakan setiap nilai fitur ke dalam indeks bin yang sesuai:

$$5 \rightarrow bin\ 0$$

$$6 \rightarrow bin\ 1$$

Sehingga hasil transformasi histogram *binning* untuk fitur pertama adalah:

$$x_{bin}^{(1)} = [0,1]$$

2. Fitur 2:

Nilai minimum, maksimum, dan lebar bin pada fitur kedua dihitung sebagai berikut:

$$x^{(2)} = [406.650;\ 833.627,46478873]$$

$$x_{min}^{(2)} = 406.650$$

$$x_{max}^{(2)} = 833.627,46478873$$

Lebar bin diperoleh dengan:

$$w_2 = \frac{833.627,46478873 - 406.650}{2}$$

$$w_2 = 213.488.73239437$$

Selanjutnya ditentukan batas setiap bin sebagai berikut:

$$e_0^{(2)} = 406.650$$

$$e_1^{(2)} = 406.650 + 213.488,73239437$$

$$e_1^{(2)} = 620.138,73239437$$

$$e_2^{(2)} = 833.627,46478873$$

Dengan demikian diperoleh batas bin (*edge*) dan interval sebagai berikut:

$$edge = [406.650;\ 620.138,73239437;\ 833.627,46478873]$$

$$Bin\ 0: [406.650;\ 620.138,73239437)$$

$$Bin\ 1: [620.138,73239437;\ 833.627,46478873]$$

Tahap berikutnya adalah memetakan setiap nilai fitur ke dalam indeks bin yang sesuai:

$$406.650 \rightarrow bin\ 0$$

$$833.627.46478873 \rightarrow bin\ 1$$

Sehingga hasil transformasi histogram *binning* untuk fitur kedua adalah:

$$x_{bin}^{(2)} = [0,1]$$

3. Fitur 3:

Nilai minimum, maksimum, dan lebar bin pada fitur ketiga dihitung sebagai berikut:

$$x^{(3)} = [2,2]$$
$$x_{\min}^{(3)} = 2$$
$$x_{\max}^{(3)} = 2$$

Karena seluruh nilai pada fitur ini sama, proses histogram dengan pembagian interval tetap menghasilkan batas bin sebagai berikut:

$$edge = [1,5;\ 2; 2,5]$$

Sehingga interval bin yang terbentuk adalah:

$$Bin\ 0: [1,5; 2)$$
$$Bin\ 1: [2; 2,5]$$

Tahap berikutnya adalah memetakan setiap nilai fitur ke dalam indeks bin yang sesuai:

$$2 \rightarrow bin\ 1$$
$$2 \rightarrow bin\ 1$$

Sehingga hasil transformasi histogram *binning* untuk fitur ketiga adalah:

$$x_{bin}^{(3)} = [1,1]$$

**Matriks Hasil Histogram *Binning***

Setelah seluruh fitur dipetakan ke bin masing-masing, diperoleh matriks fitur baru

$$X_{bin} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Matriks ini memiliki dimensi yang sama dengan dataset awal, namun setiap elemen kini merepresentasikan indeks bin dari nilai fitur tersebut.

**Langkah 4: Iterasi langkah 5-6 sebanyak M kali, di mana M=1 sebagai contoh**

**Langkah 5: Hitung *gradient* dan *hessian***

$$g = F(x) - y$$
$$g = \begin{bmatrix} 817.228,60225738 \\ 817.228,60225738 \end{bmatrix} - \begin{bmatrix} 833.627,46478873 \\ 800.829,73972603 \end{bmatrix}$$
$$g = \begin{bmatrix} -16.398.86253135 \\ 16.398.86253135 \end{bmatrix}$$

$$h = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Nilai $g_i$ dan $h_i$ untuk setiap data direpresentasikan dalam bentuk matriks agar penulisannya lebih ringkas. Setiap elemen pada matriks tersebut tetap merepresentasikan nilai turunan pada data ke-$i$, sehingga elemen pertama pada matriks $g$ menyatakan $g_1$dan elemen pertama pada matriks $h$ menyatakan $h_1$.

**Langkah 6: Membangun *Weak Learner***

Karena jumlah data pada contoh ini sangat sedikit, seluruh bin pada histogram untuk setiap fitur digunakan. Representasi bin dari data tersebut dinyatakan sebagai $X_{bin}$.

$$X_{bin} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Berarti data yang dipakai:

$$X = \begin{bmatrix} 5 & 406.650 & 2 \\ 6 & 833.627,46478873 & 2 \end{bmatrix}$$

Berdasarkan representasi tersebut, data asli yang digunakan dalam perhitungan adalah:

$$g = \begin{bmatrix} -16.398.,6253135 \\ 16.398,86253135 \end{bmatrix}$$

$$h = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Selanjutnya dipilih sampel dengan nilai gradien tertinggi dan terendah. Pada contoh ini jumlah data hanya dua sehingga parameter sampling digunakan sebesar $a = 0,5$ dan $b = 0,5$. Dalam praktiknya, nilai tersebut umumnya ditetapkan lebih kecil karena jumlah data jauh lebih banyak.

Gradien dengan nilai terbesar berasal dari bin 1 yaitu,

$$g_1 = 16.398,86253135$$

Sementara itu, gradien dengan nilai terendah berasal dari bin 0, yang kemudian dikalikan dengan faktor penyesuaian $(1-a/b)$, sehingga diperoleh:

$$g_0 = -16.398,86253135 \times \frac{1 - 0,5}{0,5}$$

$$= -16.398,86253135$$

Dilakukan iterasi pada setiap bin di tiap fitur untuk menentukan bin terbaik, di mana bin memiliki fungsi yang serupa dengan titik pemisah $s$ pada *decision tree*.

Misalnya dengan $\lambda$=1 dan $\gamma$=0:

1. *Node split* pada Fitur 1, Bin 1:

*Left child* (bin < 1) yang terdiri dari bin 0 (dengan nilai fitur *month* sebesar 5):

$$\sum_{i \in I_L} g_i = -16.398,86253135$$

$$\sum_{i \in I_L} h_i + \lambda = 1 + 1 = 2$$

*Right child* (bin ≥ 1) terdiri dari bin 1 (dengan nilai fitur *month* sebesar 6):

$$\sum_{i \in I_R} g_i = 16.398{,}86253135$$

$$\sum_{i \in I_R} h_i + \lambda = 1 + 1$$

*Original node*:

$$\sum_{i \in I} g_i = -16.398{,}86253135 + 16.398{,}86253135 = 0$$

$$\sum_{i \in I} h_i + \lambda = 1 + 1$$

Gain dihitung menggunakan rumus gain:

$$\text{Gain} = \frac{1}{2}\left(\frac{\left(\sum_{i \in I_L} g_i\right)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{\left(\sum_{i \in I_R} g_i\right)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{\left(\sum_{i \in I} g_i\right)^2}{\sum_{i \in I} h_i + \lambda}\right) - \gamma$$

$$= \frac{1}{2}\left(\frac{(-16.398{,}86253135)^2}{2} + \frac{(16.398{,}86253135)^2}{2} - \frac{(-0)^2}{3}\right) - 0$$

$$= \frac{134.461.346{,}161099 + 134.461.346{,}161097 - 0}{2} - 0$$

$$= 134.461.346{,}161098$$

2. *Node split* pada Fitur 2, Bin 1:

*Left child* (bin < 1) terdiri dari bin 0 (dengan nilai fitur lag_1 sebesar 406.650):

$$\sum_{i \in I_L} g_i = -16.398{,}86253135$$

$$\sum_{i \in I_L} h_i + \lambda = 1 + 1 = 2$$

*Right child* (bin ≥ 1) terdiri dari bin 1 (dengan nilai fitur lag_1 sebesar 833627,4647887324):

$$\sum_{i \in I_R} g_i = 16.398{,}86253135$$

$$\sum_{i \in I_R} h_i + \lambda = 1 + 1 = 2$$

Original node:

$$\sum_{i \in I} g_i = -0$$

$$\sum_{i \in I} h_i + \lambda = 2 + 1 = 3$$

Gain dihitung menggunakan rumus gain:

$$\text{Gain} = \frac{1}{2}\left(\frac{(-16.398{,}86253135)^2}{2} + \frac{(16.398{,}86253135)^2}{2} - \frac{(-0)^2}{3}\right) - \gamma$$

$$= \frac{134.461.346{,}161099 + 134.461.346{,}161097 - 0}{2}$$

$$= 134.461.346{,}161098$$

3. *Node split* pada Fitur 3, Bin 1:

*Left child* kosong.

$$\sum_{i \in I_L} g_i = 0$$

$$\sum_{i \in I_L} h_i + \lambda = 0 + 1 = 1$$

*Right child* (bin ≥ 1) terdiri dari bin 1 (dengan nilai fitur *bundle_1* sebesar 2):

$$\sum_{i \in I_R} g_i = -0$$

$$\sum_{i \in I_R} h_i + \lambda = 2 + 1 = 3$$

*Original node*:

$$\sum_{i \in I} g_i = -0$$

$$\sum_{i \in I} h_i + \lambda = 2 + 1 = 3$$

*Gain* dihitung menggunakan rumus gain:

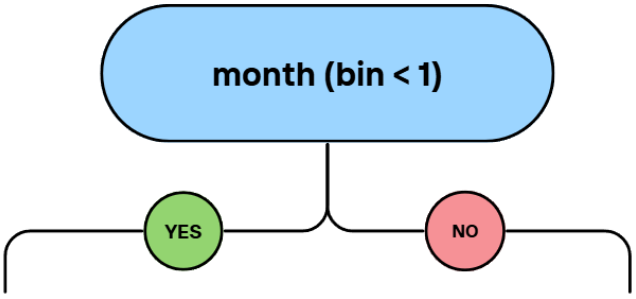
$$\text{Gain} = \frac{1}{2}\left(\frac{(0)^2}{1} + \frac{(-0)^2}{3} - \frac{(-0)^2}{3}\right) - 1$$

$$= \frac{0 + 0 - 0}{2} - 1 = 0$$

Berdasarkan perhitungan *gain* yang diperoleh, pemisahan terbaik terjadi pada fitur ke-1 dengan kondisi *split* pada bin < 1. Pemisahan ini kemudian digunakan untuk menentukan pembagian indeks data ke *node* kiri dan kanan pada

tahap selanjutnya. *Split* tersebut menghasilkan nilai gain sebesar 134.461.346,161098 yang menunjukkan pengurangan *loss* model sebesar nilai tersebut.

Gambar 2.15 *Weak Learner Root*




Berdasarkan split pada fitur *month* dengan kondisi bin < 1, data terbagi menjadi dua node. *Left child* berisi data dengan indeks 0, sedangkan *right child* berisi data dengan indeks 1. Karena masing-masing *node* hanya memiliki satu data, maka tidak dilakukan proses *split* lebih lanjut dan kedua *node* tersebut menjadi *leaf node*.

Pada kondisi umum dengan jumlah data yang lebih besar, proses pembentukan pohon keputusan tidak berhenti pada satu *split* saja. Algoritma LightGBM akan menghitung nilai *gain* dari setiap kemungkinan *split* pada seluruh cabang (*branch*) yang tersedia. Cabang dengan nilai *gain* terbesar akan dipilih untuk dilakukan *split*. Setelah *split* dilakukan, cabang tersebut akan menghasilkan dua cabang baru yang kembali dievaluasi nilai *gain*-nya bersama dengan cabang-cabang lainnya. Proses ini dilakukan secara berulang dengan selalu memilih *split* yang menghasilkan *gain* terbesar hingga mencapai kondisi penghentian, sehingga seluruh *node* yang tersisa menjadi *leaf node*.

Nilai *output* untuk *leaf node* kiri dan kanan ditetapkan sebagai berikut (dengan *leaf* kiri merupakan *leaf* ke-1 dan *leaf* kanan leaf ke-2):
Leaf kiri (leaf ke-1):

Karena *node* kiri terbentuk dari kondisi pemisahan bin < 1, maka nilai gradien dan hessian yang digunakan dalam perhitungan adalah nilai yang berasal dari data pada bin tersebut, yaitu $g_0$dan $h_0$. Nilai output pada leaf node kemudian dihitung menggunakan rumus nilai leaf pada LightGBM sebagai berikut:

$$w_1^* = -\frac{\sum_{i \in I_1} g_i}{\sum_{i \in I_1} h_i + \lambda}$$

Berdasarkan data yang terdapat pada *node* kiri, diperoleh:

$$\sum_{i \in I_1} g_i = -16.398{,}86253135$$

$$\sum_{i \in I_1} h_i + \lambda = 1 + 1 = 2$$

Sehingga nilai *output* pada *leaf node* kiri dapat dihitung sebagai berikut:

$$w_1^* = -\frac{-16.398{,}86253135}{2}$$

$$w_1^* = 8.199{,}431265675$$

Dengan demikian, nilai prediksi yang dihasilkan oleh *leaf node* kiri adalah sebesar 8.199,431265675.

*Leaf* Kanan (leaf ke-2):

Karena node kanan terbentuk dari kondisi pemisahan bin ≥ 1, maka nilai gradien dan hessian yang digunakan dalam perhitungan berasal dari data pada bin tersebut, yaitu $g_1$dan $h_1$. Nilai output pada leaf node dihitung menggunakan rumus nilai leaf pada LightGBM sebagai berikut:

$$w_2^* = -\frac{\sum_{i \in I_2} g_i}{\sum_{i \in I_2} h_i + \lambda}$$

Berdasarkan data yang terdapat pada *node* kanan, diperoleh:

$$\sum_{i \in I_2} g_i = 16.398{,}86253135$$

$$\sum_{i \in I_2} h_i + \lambda = 1 + 1 = 2$$

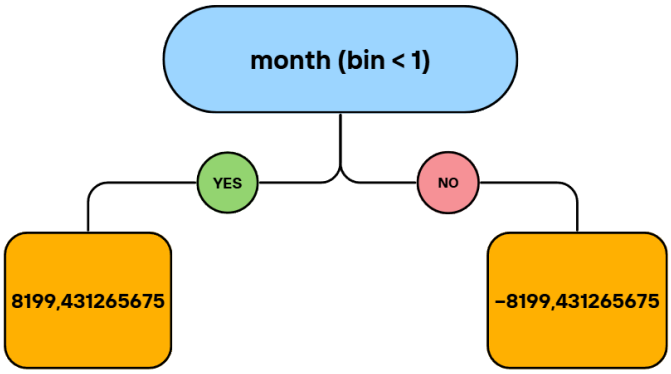
Sehingga nilai *output* pada *leaf node* kanan dapat dihitung sebagai berikut:

$$w_2^* = -\frac{16.398,86253135}{2}$$

$$w_2^* = -8.199,431265675$$

Dengan demikian, nilai prediksi yang dihasilkan oleh *leaf node* kanan adalah sebesar −8.199,431265675.

Gambar 2.16 Hasil Akhir *Weak Learner*




**Langkah 7: Perbarui model**

Persamaan model pada iterasi ke-$m$ dinyatakan sebagai:

$$F_m(x) = F_{m-1}(x) + \nu h(x; a_m)$$

Sebagai contoh untuk iterasi pertama:

$$F_1 = F_0 + 0,1\, h(x; a_1)$$

$$F_1 = 817.228,6022573798 + 0,1\, h(x; a_1)$$

Pada contoh ini, hanya dilakukan pembaruan pada satu iterasi karena M=1, namun dalam praktiknya nilai M biasanya lebih besar dari satu. Model dilatih secara bertahap dengan meminimalkan *loss* yang dihasilkan pada $F_m(x)$ untuk data baru, sehingga setiap langkah pelatihan bertujuan memperbarui *split* dan *leaf* pada pohon agar pengurangan *loss* menjadi maksimal, sesuai prosedur yang dijelaskan pada langkah 6.

**Langkah 8: Akhiri perulangan**

Perulangan pada proses pelatihan dihentikan karena pada contoh ini $M = 1$, sebagaimana telah dijelaskan sebelumnya. Untuk proses inferensi, misalkan terdapat data numerik sebagai berikut:

Tabel 2.23 Data *Testing* Numerik

| *month* | *lag_1* |
|---|---|
| 7 | 800.829,739726 |

Serta data *sparse* untuk fitur tentang *season*:

Tabel 2.24 Data Testing Sparse

| *season_Low* | *season_Mid* | *season_High* | *season_Peak* |
|---|---|---|---|
| 0 | 0 | 1 | 0 |

Data tersebut kemudian dimasukkan ke dalam prosedur *Merge Exclusive Feature*, karena *bundling* dan rentang bin (*bin range*) sudah diketahui, yaitu:

$$bundle = [[1,2,3,4]]$$

$$binRange = [0, 1, 2, 3, 4]$$

Nilai pertama yang tidak nol terdapat pada fitur ketiga, sehingga diperoleh:

$$val = fitur + binRange[3] = 1 + 3 = 4$$

Sehingga data baru yang digunakan untuk prediksi adalah:

Tabel 2. 25 Data *Testing* LightGBM

| *month* | *lag_1* | *bundle_1* |
|---|---|---|
| 7 | 800.829,739726 | 4 |

Selanjutnya, *edge* yang telah dihitung per kolom digunakan untuk menentukan bin masing-masing fitur:

1. Fitur 1 (month): $[5; 5{,}5; 6]$
2. Fitur 2 (lag_1): $[406.650; 620.138{,}73239437; 833.627{,}46478873]$

3. Fitur 3 (bundle_1): $[1{,}5; 2; 2{,}5]$

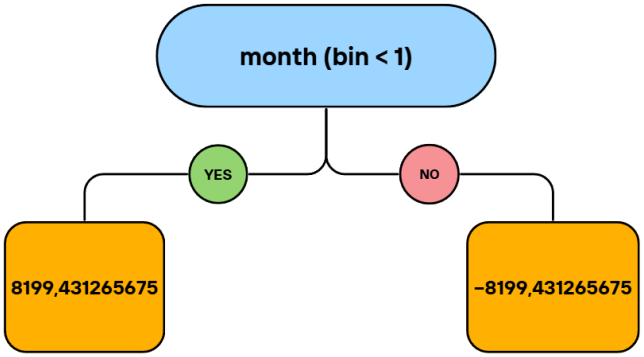
Data yang telah dipetakan ke dalam bin kemudian dimasukkan ke setiap model *weak learner*. Karena hanya terdapat satu observasi, pemetaan bin adalah sebagai berikut:

1. Fitur 1: 7 → bin 1
2. Fitur 2: 800.829,739726 → bin 1
3. Fitur 3: 800.829,739726 → bin 2

Setelah seluruh fitur dipetakan ke bin masing-masing, diperoleh matriks fitur baru $X_{\text{bin}}$, sebagai berikut:

$$X_{bin} = [\,1 \quad 1 \quad 2\,]$$

Gambar 2.17 *Weak Learner Decision* Tree




Selanjutnya, prediksi menggunakan *weak learner* berupa *decision tree* dilakukan. Karena fitur utama yang dipilih adalah *month* (fitur 1) dan *bin*-nya adalah 1, prediksi *leaf node* tersebut adalah $-8.199{,}431265675$. Nilai ini kemudian dimasukkan ke dalam formula LightGBM untuk menghitung output model:

$$\begin{aligned} F_1(x) &= F_0 + \nu \cdot h(x; a_1) \\ &= 817.228{,}6022573798 + 0{,}1 \times (-8.199{,}431265675) \\ &= 816.408{,}6591308123 \end{aligned}$$

Dengan demikian, hasil prediksi untuk harga (*price*) pada bulan ke-7 adalah sebesar 816.408,6591308123. Hasil ini mencerminkan penerapan *LightGBM*, yang merupakan model *gradient boosting* yang bertujuan mengoptimalkan proses

pelatihan sambil mempertahankan performa prediksi yang baik. Teknik ini menggunakan *histogram binning*, *exclusive feature bundling (EFB)*, serta *weak learner* berupa *decision tree* dengan strategi pembaruan *leaf-wise* untuk memaksimalkan pengurangan *loss* secara efisien.

### 2.2.6 Evaluasi Kinerja Model

Menilai sejauh mana performa sebuah model bekerja tampaknya menjadi salah satu langkah esensial ketika merancang sistem peramalan harga kamar hotel. Pengujian ini diperlukan guna menakar seberapa dekat tebakan algoritma terhadap tarif aktual yang terekam secara historis pada saluran Booking.com. Dalam upaya mengukur tingkat akurasi model LightGBM yang akan memproses fitur-fitur berdimensi tinggi seperti identitas properti dan kamar, penelitian ini memanfaatkan beberapa tolok ukur statistik. Penggunaan metrik-metrik tersebut bukan sekadar pemenuhan syarat matematis, melainkan ditujukan untuk menggali interpretasi yang lebih mendalam mengenai letak dan besaran simpangan prediksi yang dihasilkan.

1. *Mean Absolute Error* (MAE)

Secara konsep, MAE mencoba mengukur rerata jarak absolut antara nilai yang diproyeksikan oleh algoritma dengan harga riil di lapangan. Karena sifatnya yang murni absolut, arah simpangannya, baik itu saat model menebak harga terlalu tinggi maupun terlalu rendah, tidak akan memengaruhi perhitungan akhir. Tolok ukur ini diduga sangat relevan apabila kita ingin melihat seberapa besar melesetnya harga peramalan dalam satuan aslinya secara gamblang, tanpa memberikan penalti berlebihan pada kesalahan tebakan tertentu (Sanjaya, 2024).

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{2.57}$$

Keterangan:

$\hat{y}_i$ : Nilai hasil prediksi harga

$y_i$ : Nilai aktual harga

$n$ : Jumlah data sampel

**Contoh 2.6**

Misalkan kita memiliki data hasil prediksi harga kamar yang diperoleh dari model LightGBM (y_*pred*), dan nilai aktualnya (y_test) pada beberapa periode tertentu sebagai berikut:

Tabel 2.26 Dataset MAE

| *y_test* | *y_pred* |
|---|---|
| 763.482,873735 | 803.740,181420 |
| 910.959,112854 | 795.846,059310 |
| 1.147.920,130773 | 896.153,442572 |
| 1.152.168,977191 | 896.153,442572 |
| 1.087.690,743703 | 896.153,442572 |

Rata-rata kesalahan absolutnya dihitung sebagai:

$$\begin{aligned}
\text{MAE} &= \frac{1}{5}\sum_{i=1}^{5}|y_i - \hat{y}_i| \\
&= \frac{1}{5}(|763.482{,}873735 - 803.740{,}181420| \\
&\qquad + |910.959{,}112854 - 795.846{,}059310| \\
&\qquad + |1.147.920{,}130773 - 896.153{,}442572| \\
&\qquad + |1.152.168{,}977191 - 896.153{,}442572| \\
&\qquad + |1.087.690{,}743703 - 896.153{,}442572|) \\
&= 170.937{,}97703588434
\end{aligned}$$

Hasil perhitungan MAE sebesar 170.938 menunjukkan bahwa, secara rata-rata, prediksi harga model menyimpang sekitar 170.938 rupiah dari harga aktual. Nilai MAE memberikan ukuran yang sederhana dan langsung untuk menilai kesalahan prediksi karena menggunakan selisih absolut tanpa mengkuadratkan perbedaan seperti pada RMSE. Semakin kecil nilai MAE, semakin akurat prediksi model terhadap nilai riil.

2. *Mean Squared Error* (MSE)

Pendekatan lain yang lazim diaplikasikan adalah MSE. Metrik ini beroperasi agak berbeda karena sengaja mengkuadratkan selisih kesalahan prediksi sebelum menghitung nilai rata-ratanya. Akibat proses pengkuadratan tersebut, model secara otomatis akan mendapat "hukuman" yang lebih berat jika melakukan kesalahan tebakan dalam jumlah besar. Mengingat sifatnya yang cukup sensitif terhadap data pencilan (*outlier*), MSE agaknya cocok diterapkan untuk memastikan algoritma tidak mengeluarkan proyeksi harga yang melenceng terlalu drastis dari batas kewajaran, yang mana hal itu berisiko memicu hilangnya potensi pendapatan (Sanjaya, 2024).

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{2.58}$$

Keterangan:

$n$ : Jumlah data pengamatan

$y_i$ : Nilai aktual harga

$\hat{y}_i$ : Nilai hasil prediksi harga

**Contoh 2.7**

Dengan menggunakan data pengujian yang sama:

Tabel 2.27 Dataset MSE

| *y_test* | *y_pred* |
|---|---|
| 763.482,873735 | 803.740,181420 |
| 910.959,112854 | 795.846,059310 |
| 1.147.920,130773 | 896.153,442572 |
| 1.152.168,977191 | 896.153,442572 |
| 1.087.690,743703 | 896.153,442572 |

MSE dihitung sebagai rata-rata kuadrat selisih antara nilai aktual dan prediksi, sehingga diperoleh:

$$\begin{aligned}
\text{MSE} &= \frac{1}{5}\sum_{i=1}^{5}(y_i - \hat{y}_i)^2 \\
&= \frac{1}{5}((763.482.873735 - 803.740.181420)^2 \\
&\qquad + (910.959.112854 - 795.846.059310)^2 \\
&\qquad + (1.147.920.130773 - 896.153.442572)^2 \\
&\qquad + (1.152.168.977191 - 896.153.442572)^2 \\
&\qquad + (1.087.690.743703 - 896.153.442572)^2) \\
&= 36.097.724.579{,}2773
\end{aligned}$$

Berdasarkan perhitungan ini, dapat disimpulkan bahwa rata-rata kuadrat kesalahan prediksi terhadap nilai aktual cukup besar, yaitu sekitar 36 miliar. MSE memberikan gambaran seberapa besar kesalahan prediksi yang bersifat kuadrat, sehingga kesalahan yang lebih besar akan memberi kontribusi lebih signifikan terhadap nilai akhir. Metode ini berguna untuk menilai performa model secara menyeluruh dan menekankan dampak prediksi yang menyimpang jauh dari nilai aktual.

3. *Root Mean Squared Error* (RMSE)

Meski MSE dinilai ampuh untuk meredam kesalahan berskala besar, hasil pengukurannya berada dalam format kuadrat yang kemungkinan besar akan menyulitkan proses interpretasi. Untuk menyiasati hal tersebut, RMSE mengambil nilai akar kuadrat dari perhitungan MSE untuk mengembalikan satuan hasil evaluasi ke bentuk awalnya (Sanjaya, 2024). Sebagai contoh, jika perhitungan data target berbasis nominal rupiah, hasil luaran RMSE juga akan berada dalam besaran rupiah, sehingga memudahkan pihak manajemen operasional untuk menakar masuk akalnya sebuah peramalan.

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{2.59}$$

Keterangan:

$n$ : Jumlah data pengamatan

$y_i$ : Nilai aktual harga

$\hat{y}_i$ : Nilai hasil prediksi harga

**Contoh 2.8**

Dengan menggunakan data pengujian yang sama:

Tabel 2.28 Dataset RMSE

| *y_test* | *y_pred* |
|---|---|
| 763.482,873735 | 803.740,181420 |
| 910.959,112854 | 795.846,059310 |
| 1.147.920,130773 | 896.153,442572 |
| 1.152.168,977191 | 896.153,442572 |
| 1.087.690,743703 | 896.153,442572 |

RMSE dihitung sebagai akar kuadrat dari nilai MSE, sehingga diperoleh nilai kesalahan yang kembali berada pada satuan yang sama dengan data asli:

$$\begin{aligned}\sum_{i=1}^{5}(y_i - \hat{y}_i)^2 &= ((763.482{,}873735 - 803.740{,}181420)^2 \\ &+ (910.959{,}112854 - 795.846{,}059310)^2 \\ &+ (1.147.920{,}130773 - 896.153{,}442572)^2 \\ &+ (1.152.168{,}977191 - 896.153{,}442572)^2 \\ &+ (1.087.690{,}743703 - 896.153{,}442572)^2) \\ &= 180.488.622.896{,}38647\end{aligned}$$

$$\text{RMSE} = \sqrt{\frac{1}{5}\sum_{i=1}^{5}(y_i - \hat{y}_i)^2}$$

$$= \sqrt{\frac{1}{5}180.488.622.896{,}38647}$$

$$= 189.994{,}0119563701$$

Berdasarkan hasil perhitungan tersebut, nilai RMSE sebesar 189.994,0119563701 menunjukkan bahwa rata-rata kesalahan prediksi terhadap nilai aktual berada pada kisaran sekitar 189.994 satuan harga. RMSE memberikan ukuran kesalahan yang mempertimbangkan kuadrat selisih antara nilai aktual dan prediksi, sehingga kesalahan dengan nilai yang lebih besar akan memiliki pengaruh yang lebih signifikan terhadap hasil akhir perhitungan. Oleh karena itu, metrik ini sering digunakan untuk menilai tingkat akurasi model karena mampu menyoroti deviasi prediksi yang relatif besar dari nilai sebenarnya.

4. Koefisien Determinasi (R-*Squared*)

Berbeda dengan ketiga metrik sebelumnya yang berfokus menyoroti seberapa jauh melencengnya suatu prediksi, R-*Squared* atau koefisien determinasi lebih sering digunakan untuk mengevaluasi kemampuan fitur masukan dalam menjelaskan fluktuasi variabel target. Rentang nilainya bergerak dari batas 0 hingga angka 1. Apabila hasil pengujiannya semakin mendekati 1, kemungkinan besar fitur-fitur yang telah direkayasa, seperti informasi *lag* historis dan musiman sudah sangat representatif dalam menangkap pola perubahan harga kamar yang sebenarnya (Sanjaya, 2024).

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{2.60}$$

Keterangan:

$y_i$ : Nilai yang diamati (aktual) untuk pengamatan ke-i

$\hat{y}_i$ : Nilai yang diramalkan (prediksi) untuk pengamatan ke-i

$\bar{y}$ : Rata-rata dari nilai yang diamati

**Contoh 2.9**

Dengan menggunakan data pengujian yang sama:

Tabel 2. 29 Dataset R-*Squared*

| *y_test* | *y_pred* |
|---|---|
| 763.482,873735 | 803.740,181420 |
| 910.959,112854 | 795.846,059310 |
| 1.147.920,130773 | 896.153,442572 |
| 1.152.168,977191 | 896.153,442572 |
| 1.087.690,743703 | 896.153,442572 |

Koefisien determinasi $R^2$ diperoleh dengan membandingkan jumlah kuadrat kesalahan prediksi dengan jumlah kuadrat penyimpangan data aktual terhadap nilai rata-ratanya sebagai berikut:

$$\begin{aligned}\sum_{i=1}^{5}(y_i - \hat{y}_i)^2 \;=\; &((763.482{,}873735 - 803740{,}181420)^2 \\ &+ (910.959{,}112854 - 795.846{,}059310)^2 \\ &+ (1.147.920{,}130773 - 896.153{,}442572)^2 \\ &+ (1.152.168{,}977191 - 896.153{,}442572)^2 \\ &+ (1.087.690{,}743703 - 896.153{,}442572)^2) \\ &= 180.488.622.896{,}38647\end{aligned}$$

$$\begin{aligned}\bar{y} = \frac{1}{5} \times (&763.482{,}873735 + 910.959{,}112854 + 1.147.920{,}130773 \\ &+ 1.152.168{,}977191 + 1.087.690{,}743703) \\ = \;&1.012.444{,}3676512002\end{aligned}$$

$$\begin{aligned}\sum_{i=1}^{5}(y_i - \bar{y})^2 = &(763.482{,}873735 - 1.012.444{,}3676512002)^2 \\ &+ (910.959{,}112854 - 1.012.444{,}3676512002)^2 \\ &+ (1.147.920{,}130773 - 1.012.444{,}3676512002)^2 \\ &+ (1.152.168{,}977191 - 1.012.444{,}3676512002)^2 \\ &+ (1.087.690{,}743703 - 1.012.444{,}3676512002)^2 \\ &= 115.819.748.407{,}65121\end{aligned}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{5}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{5}(y_i - \bar{y})^2}$$
$$= 1 - \frac{180.488.622.896,38647}{115.819.748.407,65121}$$
$$= -0,5583579258100266$$

Berdasarkan hasil perhitungan tersebut, diperoleh nilai $R^2$ sebesar -0,5583579258100266 yang menunjukkan bahwa model tidak mampu menjelaskan variasi data aktual dengan baik. Nilai $R^2$yang bernilai negatif mengindikasikan bahwa performa prediksi model lebih buruk dibandingkan dengan pendekatan sederhana yang menggunakan nilai rata-rata data sebagai prediksi. Dengan demikian, nilai ini menunjukkan bahwa hubungan antara nilai prediksi dan nilai aktual pada data pengujian masih lemah sehingga tingkat akurasi model dalam menjelaskan variasi data relatif rendah.

5. *Mean Absolute Percentage Error* (MAPE)

Selain pengukuran absolut, terdapat pula perhitungan MAPE yang menawarkan sudut pandang evaluasi berbasis persentase. Dalam konteks operasional industri perhotelan, merangkum rasio kesalahan ke dalam bentuk persen umumnya terasa lebih intuitif bagi pengambil keputusan. Mengingat harga dasar tiap tipe kamar sangat bervariasi, selisih nominal beberapa ratus ribu rupiah mungkin berdampak signifikan pada akurasi kamar tipe standar, tetapi terasa wajar untuk tipe vila mewah. MAPE berupaya menetralkan bias bawaan ini dengan memaparkan besaran simpangan secara proporsional terhadap harga aktualnya (Raafi Herlando dkk., 2025). Nilai persentase yang kian mengecil menandakan bahwa tingkat kepercayaan peramalan model semakin tinggi.

$$MAPE = \frac{1}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \times 100 \tag{2.61}$$

Keterangan:

$y_i$ : Nilai aktual data ke-i

$\hat{y}_i$ : Nilai prediksi data ke-i

$n$ : Jumlah data

Kombinasi dari kelima metrik pengujian tersebut tampaknya sanggup menyajikan evaluasi yang cukup menyeluruh terhadap performa prediksi. Hasil pengukuran ini nantinya akan memandu proses kalibrasi algoritma LightGBM, sebelum sistem peramalan ini diserahkan dan difungsikan sebagai landasan penentuan harga dinamis bagi perusahaan.

**Contoh 2.10**

Dengan menggunakan data pengujian yang sama:

Tabel 2.30 Dataset MAPE

| *y_test* | *y_pred* |
|---|---|
| 763.482,873735 | 803.740,181420 |
| 910.959,112854 | 795.846,059310 |
| 1.147.920,130773 | 896.153,442572 |
| 1.152.168,977191 | 896.153,442572 |
| 1.087.690,743703 | 896.153,442572 |

MAPE dihitung sebagai rata-rata persentase kesalahan antara nilai aktual dan nilai prediksi, sehingga diperoleh:

$$\begin{aligned}\sum_{i=1}^{n}\left|\frac{y_i-\hat{y}_i}{y_i}\right| &= \left(\left|\frac{763.482,873735-803.740,181420}{763.482,873735}\right|\right.\\ &+\left|\frac{910.959,112854-795.846,059310}{910.959,112854}\right|\\ &+\left|\frac{114.7920,130773-896.153,442572}{1147920,130773}\right|\\ &+\left|\frac{1.152.168,977191-896.153,442572}{1152168,977191}\right|\\ &+\left.\left|\frac{1.087.690,743703-896.153,442572}{1087690,743703}\right|\right)\\ &= 0,7967158799288666\end{aligned}$$

$$MAPE = \frac{1}{5} 0{,}7967158799288666 \times 100$$
$$= 15{,}934317598577334$$

Berdasarkan hasil perhitungan tersebut, dapat disimpulkan bahwa rata-rata persentase kesalahan antara nilai prediksi dan nilai aktual adalah sekitar 15,93%. MAPE memberikan gambaran mengenai tingkat kesalahan prediksi dalam bentuk persentase terhadap nilai aktual, sehingga memudahkan interpretasi terhadap akurasi model. Semakin kecil nilai MAPE yang diperoleh, maka semakin baik kemampuan model dalam menghasilkan prediksi yang mendekati nilai sebenarnya.

# BAB III
# TINJAUAN UMUM INSTANSI

## 3.1 Gambaran Umum

PT Genius Utama Indonesia, yang beroperasi menggunakan nama dagang Genius Hospitality Management & Development (GHM), adalah sebuah badan usaha swasta yang berfokus pada sektor manajemen perhotelan. Bidang usaha perusahaan ini mencakup pembangunan fasilitas, pengelolaan operasional akomodasi, serta pengembangan infrastruktur pariwisata yang berkelanjutan. Wilayah operasi utama perusahaan berada di area kepulauan Provinsi Nusa Tenggara Barat.

Secara legalitas, perusahaan ini berdiri sebagai Perseroan Terbatas (PT) dan terdaftar secara resmi dengan nomor AHU-02000241.AH.01.11 TAHUN 2023. Kantor operasional PT Genius Utama Indonesia berlokasi di Jalan Kartini, Dusun Gili Air, Kecamatan Pemenang, Kabupaten Lombok Utara, Provinsi Nusa Tenggara Barat.

Kegiatan operasional perusahaan meliputi siklus penuh dalam pengembangan properti wisata. Proses tersebut dimulai dari tahapan identifikasi potensi pariwisata di lokasi strategis, pelaksanaan konstruksi bangunan, hingga manajemen operasional harian guna menjaga standar kualitas layanan penginapan. Di samping itu, perusahaan juga mengelola pemasaran dan strategi pendapatan (*revenue management*) untuk mengoptimalkan tingkat hunian kamar. Sejauh ini, perusahaan telah mengelola dan menjalin kemitraan dengan berbagai properti, di

antaranya Kaluku Gili Resort, Villa Phyphy, Pandan Bungalow, dan beberapa fasilitas akomodasi lainnya di kawasan Lombok dan sekitarnya.

Gambar 3.1 Kaluku Gili Resort – PT Genius Utama Indonesia

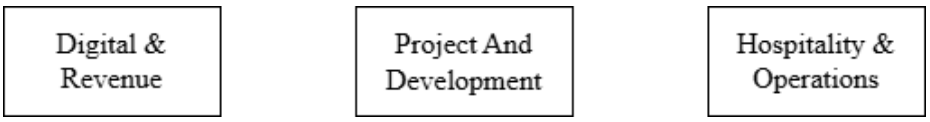
Sebagai pedoman dalam menjalankan kegiatan usahanya, PT Genius Utama Indonesia memiliki visi dan misi perusahaan sebagai berikut.

**Visi:** Menjadi pilihan utama bagi wisatawan kelas atas yang mencari pengalaman kemewahan dan relaksasi di kawasan kepulauan, serta menjadi merek hospitalitas terkemuka yang mampu menciptakan pengalaman menginap yang berkesan bagi para tamu.

**Misi:** Mendominasi pasar perhotelan di kawasan Gili Islands dengan menyediakan akomodasi berkualitas, layanan yang bersifat personal, serta pengalaman tamu yang optimal melalui inovasi dan pengembangan kemitraan strategis.

## 3.2 Struktur Organisasi

Struktur organisasi di PT Genius Utama Indonesia disusun secara ringkas untuk memastikan setiap tahapan bisnis, mulai dari pengembangan properti wisata hingga optimalisasi pendapatan, dapat berjalan selaras. Pembagian wewenang dalam perusahaan ini difokuskan pada tiga divisi utama yang dipimpin oleh tenaga ahli dengan spesialisasi masing-masing. Susunan hierarki dan pembagian tugas tersebut dapat dilihat pada Gambar 3.2.




Gambar 3.2 Struktur Organisasi PT Genius Utama Indonesia

Setiap divisi memiliki peran dan tanggung jawab operasional yang spesifik, sehingga tercipta koordinasi yang efektif dalam pengelolaan berbagai properti seperti hotel dan vila. Berikut adalah penjelasan mengenai susunan struktur organisasi yang diterapkan pada PT Genius Utama Indonesia:

1. *Digital* & *Revenue* Divisi ini dikelola oleh Bapak Edi Rahmadi, yang bertugas merancang strategi pemasaran digital, manajemen pendapatan (*revenue management*), *e-commerce*, hingga analisis data. Bagian ini memegang peranan krusial dalam menentukan harga sewa yang kompetitif di berbagai saluran pemesanan, khususnya Booking.com. Oleh karena itu, penelitian kerja praktik yang berfokus pada peramalan harga kamar menggunakan algoritma LightGBM dirancang secara spesifik untuk mendukung tugas divisi ini.
2. *Project* & *Development* Dipimpin oleh Bapak M. Kadri, S.T., divisi ini bertanggung jawab atas seluruh proses fisik dan infrastruktur properti. Tugas utamanya mencakup perancangan desain bangunan hotel atau vila, manajemen proyek konstruksi, pemeliharaan fasilitas secara berkala, hingga pengelolaan efisiensi anggaran pembangunan.
3. *Hospitality* & *Operations* Bagian ini diarahkan oleh Bapak Andrian Yosaredo, dengan fokus pada standar layanan tamu dan operasional harian. Tanggung jawabnya meliputi manajemen keramahtamahan, kepemimpinan operasional staf di lapangan, pelayanan pelanggan, tata kelola keuangan dasar, serta eksekusi rencana strategis penjualan.

Melalui pembagian tugas yang terarah ini, proses pengambilan keputusan strategis perusahaan menjadi lebih terukur. Khususnya bagi divisi *Digital* & *Revenue*, hasil dari pemodelan prediksi harga kamar berbasis deret waktu ini diharapkan dapat memberikan acuan data yang konkret. Dengan demikian, instansi

memperoleh kemudahan dalam menetapkan rata-rata harga kamar per tipe di masa mendatang tanpa harus bergantung sepenuhnya pada analisis manual, yang pada akhirnya dapat mengoptimalkan pemasukan (*occupancy* dan ADR) bagi PT Genius Utama Indonesia.

### 3.3 Produk / Jasa

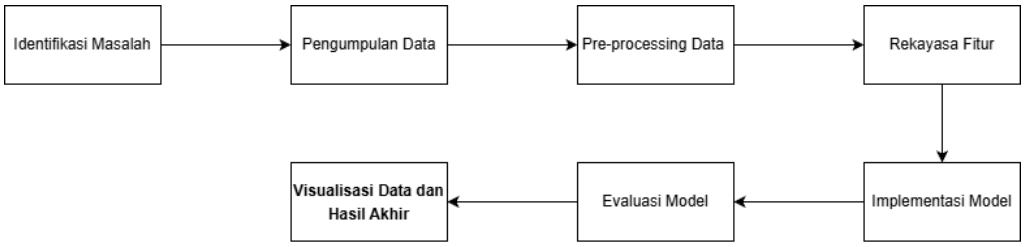
PT Genius Utama Indonesia bergerak di bidang manajemen operasional dan pemasaran properti pariwisata, dengan menitikberatkan pada penerapan strategi manajemen pendapatan (*revenue management*). Layanan operasional ini mencakup siklus pengelolaan tarif secara menyeluruh, yang dimulai dari tahap kalkulasi biaya dasar kamar, analisis kompetisi pasar, hingga penyesuaian harga secara dinamis untuk merespons fluktuasi permintaan tamu dan pergantian musim. Sebagai bentuk implementasi pemasaran, perusahaan mendistribusikan sekaligus memantau pergerakan harga akomodasi tersebut melalui berbagai agen perjalanan daring, dengan platform Booking.com sebagai salah satu saluran distribusi utama. Keseluruhan aktivitas pemeliharaan tarif digital tersebut pada akhirnya merekam data historis secara terperinci mengenai kinerja pemesanan dari setiap tipe kamar di berbagai properti yang dikelola. Basis data fungsional yang terbentuk inilah yang kemudian menjadi instrumen krusial dalam memfasilitasi analisis tren, sekaligus memberikan landasan empiris untuk memecahkan permasalahan kerja praktik dalam memproyeksikan pergerakan rata-rata harga kamar di masa depan.

# BAB IV
# METODE KERJA PRAKTIK

## 4.1 Tahapan Kerja Praktik

Tahapan kerja praktik menyebutkan dan menjelaskan langkah-langkah yang akan dijalankan secara detail dalam kerja praktik yang telah atau akan dilakukan. Penjelasan dapat menggunakan diagram yang dapat menggambarkan langkah-langkah kerja praktik serupa dengan tahapan atau metode pengembangan sistem. Perancangan dapat dituangkan ke dalam perancangan sederhana (model proses ataupun model data).

Adapun alur tahapan penelitian yang dirancang untuk menyelesaikan permasalahan prediksi harga kamar rata-rata ini digambarkan dalam diagram alir pada Gambar 4.1.




Gambar 4.1 Tahapan Kerja Praktik

Berikut adalah uraian mendetail mengenai tahapan-tahapan yang dilakukan dalam penelitian ini:

### 4.1.1 Identifikasi Masalah

Tahap ini merupakan langkah awal untuk memahami konteks bisnis dan permasalahan yang dihadapi oleh PT Genius Utama Indonesia (Genius Hospitality Management). Identifikasi difokuskan pada kesulitan manajemen dalam menentukan strategi harga yang tepat akibat fluktuasi permintaan dan variabilitas karakteristik antar properti. Masalah didefinisikan secara spesifik, yaitu perlunya sebuah model yang mampu memprediksi rata-rata harga kamar (*Average Room Rate*) per bulan untuk setiap tipe kamar (*room_id*) pada masing-masing properti

(*property_id*) di masa mendatang guna mendukung keputusan *dynamic pricing*.

### 4.1.2 Pengumpulan Data

Pengumpulan data dilakukan dengan mengekstraksi data transaksi historis dari *database* internal perusahaan. Mengingat adanya inkonsistensi data pada berbagai saluran pemasaran, pengambilan data dibatasi secara eksklusif pada saluran *Booking.com*. Data yang dikumpulkan mencakup variabel-variabel krusial seperti nomor pemesanan (*booking_id*), identitas properti (*property_id*), identitas tipe kamar (*room_id*), tanggal pemesanan (*booking_time*), harga (*price*), jumlah tamu dewasa (*num_adult*), dan jumlah anak (*num_child*). Data ini akan menjadi basis utama dalam proses pembelajaran model.

### 4.1.3 *Pre-processing* Data

*Pre-processing* data adalah proses yang dilakukan sebelum proses penelitian demi mendapatkan data yang layak untuk digunakan. Mengingat data mentah berupa data transaksi harian, tahapan ini sangat krusial dan meliputi langkah-langkah berikut:

1. **Pembersihan Data (*Data Cleaning*):** Dilakukan penyaringan (*filtering*) untuk memastikan hanya data dengan status saluran *Booking.com* yang diproses. Data dengan nilai yang hilang atau anomali yang tidak dapat diperbaiki akan dihapus atau disesuaikan.
2. **Konversi Tipe Data:** Variabel waktu seperti *booking_time* dikonversi menjadi format *datetime* untuk memungkinkan pengolahan deret waktu (*time series*). Variabel kategorikal seperti *property_id* dan *room_id* dipastikan formatnya untuk proses pengelompokan.
3. **Agregasi Data (*Data Aggregation*):** Data transaksi diubah menjadi data deret waktu bulanan. Dilakukan pengelompokan berdasarkan *property_id*, *room_id*, dan periode bulan (*month_date*). Variabel target *price* dihitung nilai rata-ratanya (*mean*), sedangkan variabel lain seperti jumlah tamu dan jumlah pemesanan dihitung totalnya (*sum* atau *count*).
4. **Penanganan *Missing Values*:** Untuk menjaga kontinuitas deret waktu, data harga yang kosong pada bulan tertentu diisi menggunakan metode imputasi

*forward fill* dan *backward fill*, dengan asumsi harga cenderung stabil dalam periode yang berdekatan.

### 4.1.4 Rekayasa Fitur (*Feature Engineering*)

Tahapan ini bertujuan untuk menciptakan fitur-fitur baru yang dapat meningkatkan kemampuan prediksi model *machine learning*. Proses ini dilakukan dengan mentransformasi data deret waktu yang telah diagregasi. Beberapa fitur yang dibentuk meliputi:

1. **Fitur Kalender:** Ekstraksi informasi waktu seperti tahun, bulan, dan kuartal dari variabel tanggal.
2. **Penyandian Fitur Kategorikal (*Encoding*):** Variabel *property_id* dan *room_id* diubah menjadi bentuk numerik menggunakan teknik *Label Encoding* agar dapat diproses oleh algoritma.
3. **Fitur *Lag* dan *Rolling Window*:** Pembuatan fitur berdasarkan data historis, seperti harga pada bulan sebelumnya (*lag_1*), harga dua bulan sebelumnya (*lag_2*), hingga harga tahun sebelumnya (*lag_12*). Selain itu, dihitung rata-rata bergerak (*rolling mean*) untuk menangkap tren jangka pendek, misalnya rata-rata harga dalam 3 bulan atau 6 bulan terakhir. Fitur ini memungkinkan model mempelajari pola autokorelasi pada data harga.

### 4.1.5 Implementasi Model

Implementasi model adalah proses penerapan model *machine learning* dalam sebuah penelitian untuk membangun algoritma yang dapat melakukan peramalan. Pada penelitian ini, algoritma yang digunakan adalah *Light Gradient Boosting Machine* (LightGBM). Data yang telah melalui proses rekayasa fitur dibagi menjadi data latih (*training set*) dan data uji (*testing set*) berdasarkan urutan waktu. Model dilatih untuk mempelajari pola hubungan antara fitur-fitur historis (*property_id*, *room_id*, *lags*, dll.) dengan target variabel yaitu rata-rata harga kamar (*price*).

### 4.1.6 Evaluasi Model

Setelah model dilatih, dilakukan tahap evaluasi untuk mengukur kinerja dan

akurasi prediksi model. Evaluasi dilakukan dengan membandingkan hasil prediksi model terhadap data aktual pada data uji. Metrik evaluasi yang digunakan meliputi *Mean Absolute Error* (MAE), *Mean Squared Error* (MSE), *Root Mean Squared Error* (RMSE), dan *Mean Absolute Percentage Error* (MAPE). Hasil evaluasi ini akan menentukan apakah model sudah cukup akurat untuk diimplementasikan atau memerlukan penyesuaian parameter lebih lanjut.

### 4.1.7 Visualisasi Data dan Hasil Akhir

Hasil akhir dari keseluruhan proses penelitian ini adalah model peramalan beserta analisis performa yang komprehensif. Hasil prediksi akan divisualisasikan dalam bentuk grafik perbandingan antara harga aktual dan harga prediksi (*Actual vs Predicted*) serta tren harga masa depan untuk setiap tipe kamar dan properti. Visualisasi ini disusun secara sistematis untuk memudahkan manajemen Genius Hospitality Management dalam memahami pola data dan performa model. Laporan akhir ini akan diserahkan kepada instansi sebagai landasan pendukung keputusan (*decision support*) dalam merumuskan strategi penetapan harga (*dynamic pricing*) yang lebih presisi di masa mendatang.

## 4.2 Data

Proses pengumpulan data bertujuan untuk memperoleh informasi yang komprehensif sebagai basis utama dalam pembangunan model peramalan. Data yang digunakan dalam penelitian ini diklasifikasikan sebagai data sekunder, yang meliputi:

1. Data transaksi historis pemesanan kamar yang diperoleh melalui ekstraksi *Application Programming Interface* (API) dari sistem manajemen properti beds24.com. Data ini dibatasi pada saluran Booking.com dan mencakup variabel krusial seperti identitas properti (*property_id*), tipe kamar (*room_id*), waktu pemesanan (*booking_time*), serta harga (*price*).
2. Data informasi musiman (*seasonality*) hotel yang diperoleh dari dokumentasi internal perusahaan. Data ini memuat informasi periode *Low Season*, *Mid-Season*, *High Season*, hingga *Peak Season* yang berfungsi sebagai *domain knowledge* untuk analisis tren.

### 4.3 Jadwal Kerja Praktik

Jadwal kerja praktik disusun sebagai acuan estimasi waktu pelaksanaan penelitian agar berjalan sistematis dan sesuai dengan tahapan yang direncanakan. Rincian jadwal kegiatan penelitian ini disajikan pada Tabel 4.1.

Tabel 4.1 Jadwal Kerja Praktik

| **Kegiatan** | **Februari 2026** | **Maret 2026** | **April 2026** | **Mei 2026** | **Juni 2026** | **Juli 2026** |
|---|---|---|---|---|---|---|
| Identifikasi Masalah | | | | | | |
| Pengumpulan Data | | | | | | |
| *Pre-processing* Data | | | | | | |
| Rekayasa Fitur (*Feature Engineering*) | | | | | | |
| Implementasi Model | | | | | | |
| Evaluasi Model | | | | | | |
| Visualisasi Data dan Hasil Akhir | | | | | | |
| Penyusunan Naskah Laporan: | | | | | | |
| • BAB I dan BAB II | | | | | | |
| • BAB III | | | | | | |
| • BAB IV | | | | | | |
| • BAB V | | | | | | |
| • BAB VI | | | | | | |

## DAFTAR PUSTAKA


Andy Hermawan, Aji Saputra, Nabila Lailinajma, Reska Julianti, Timothy Hartanto, & Troy Kornelius Daniel. (2025). Predicting Hotel Booking Cancellations Using Machine Learning for Revenue Optimization. *Router : Jurnal Teknik Informatika dan Terapan*, *3*(1), 37–48. https://doi.org/10.62951/router.v3i1.400

Aulady, Moch. A., Hudawi AS, A., & Arifin, Z. (2024). Improve Metode Lightgbm untuk Prediksi Harga Mobil Bekas Menggunakan Hyper-Parameter Tuning. *TRILOGI: Jurnal Ilmu Teknologi, Kesehatan, dan Humaniora*, *5*(3). https://doi.org/10.33650/trilogi.v5i3.9000

Budi Darma, P. S., & Wirawati, N. G. P. (2019). Pengaruh Pendapatan Kamar dan Biaya Komisi Penjualan Terhadap Average Room Rate Hotel di Daerah Denpasar. *E-Jurnal Akuntansi*, 2129. https://doi.org/10.24843/eja.2019.v27.i03.p18

Gonçalves, V. de S. F., & Carvalho, V. R. de. (2025). A Review of Interpretability Methods for Gradient Boosting Decision Trees. Dalam *Journal of the Brazilian Computer Society* (Vol. 31, Nomor 1, hlm. 640–654). Brazilian Computing Society. https://doi.org/10.5753/jbcs.2025.5324

Gusti Ayu Riyani Astarani, I., Gede Surya Rahayuda, I., Studi Informatika, P., Matematika dan Ilmu Pengetahuan Alam, F., Raya Kampus Udayana, J., Jimbaran, B., & Selatan, K. (2025). Analisis Perbandingan XGBoost dan LightGBM dalam Prediksi Penjualan Ritel Walmart Store Sales. *JNATIA*, *3*(4).

Indah, Y. M., Aristawidya, R., Fitrianto, A., Erfiani, E., & Jumansyah, L. M. R. D. (2025). Comparison of Random Forest, XGBoost, and LightGBM Methods for the Human Development Index Classification. *Jambura Journal of Mathematics*, *7*(1), 14–18. https://doi.org/10.37905/jjom.v7i1.28290

Lima Santos, L., Gomes, C., Malheiros, C., Crespo, C., & Bento, C. (2024). Factors Influencing Hotel Revenue Management in Times of Crisis: Towards Financial Sustainability. *International Journal of Financial Studies*, *12*(4). https://doi.org/10.3390/ijfs12040112

Montgomery, D. C., & Jennings, C. L. (2024). *Introduction to Time Series Analysis and Forecasting* (3rd ed.). John Wiley & Sons.

Naufal Malik Herlambang, & Harun Al Azies. (2026). TIME SERIES FORECASTING SAHAM PT ASTRA MENGGUNAKAN ALGORITMA AUTOREGRESSIVE INTEGRATED MOVING AVERAGE DAN PROPHET. *Rabit : Jurnal Teknologi dan Sistem Informasi Univrab*, *11*(1), 1105–1116. https://doi.org/10.36341/rabit.v11i1.7231

Pramita, D. A. K., Saraswati, N. W. S., Sandana, I. P. D., Dewi, D. A. P. R., & Krismentari, N. K. B. (2026). Forecasting Hotel Demand with Time Series Prediction Model Using Random Forest Regression. *sinkron*, *10*(1). https://doi.org/10.33395/sinkron.v10i1.15655

Raafi Herlando, M., Hadi Wijoyo, S., & Akbar, M. A. (2025). *ANALISIS PERBANDINGAN MACHINE LEARNING UNTUK MEMPREDIKSI*

*HARGA MINYAK DENGAN REGRESI LINEAR DAN SUPPORT VECTOR REGRESSION* (Vol. 9, Nomor 6). http://j-ptiik.ub.ac.id

Rahayu Dwiastuti Pujiningrum, & Sulistijanti, W. (2024). Peramalan Tingkat Penghunian Kamar Hotel Bintang Lima Provinsi Bali Menggunakan Metode ARIMA dan Fuzzy Time Series Lee. *JURNAL SOSIAL EKONOMI DAN HUMANIORA*, *10*(2), 158–166. https://doi.org/10.29303/jseh.v10i2.520

Rahman, A., Yusnita, A., & Ekawati, H. (2025). *Bulletin of Information Technology (BIT) Penerapan Algoritma XGBoost Dalam Prediksi Harga Sewa Kos Di Kota Samarinda*. *6*(4), 379–390. https://doi.org/10.47065/bit.v5i2.2304

Sanjaya, K. D. (2024). *PREDIKSI HARGA RUMAH DENGAN METODE REGRESI LINEAR DAN SUPPORT VECTOR REGRESSION DI DAERAH TEBET JAKARTA SELATAN* (Vol. 19, Nomor 2). Versi Cetak.

Truong, T. T. P., & Novák, V. (2023). Forecasting and Detection of Structural Breaks. Dalam O. Valenzuela, F. Rojas, L. J. Herrera, H. Pomares, & I. Rojas (Ed.), *Theory and Applications of Time Series Analysis and Forecasting: Selected Contributions from ITISE 2021*. Springer.

# LAMPIRAN